

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences

Maidan Garhi, New Delhi – 110068

MCSP-232 PROJECT REPORT

**AI-Based Real-Time eKYC System for
Deepfake-Free Identity Verification**

Submitted by:

Hardik Sharma

Enrolment No: 2353777736

Programme: MCA (MCAOL)

Under the Guidance of:

Anshul Sharma

AI Engineer

IDS Infotech Ltd., Chandigarh

July 2025

CERTIFICATE

This is to certify that the project report entitled “**AI-Based Real-Time eKYC System for Deepfake-Free Identity Verification**” submitted to Indira Gandhi National Open University in partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications (MCA)** is an authentic record of bonafide work carried out by **Mr. Hardik Sharma** (Enrolment No. **2353777736**) under my guidance and supervision.

The matter embodied in this report has not been submitted for the award of any other degree or diploma to this University or any other institution.

Signature of Supervisor

Name: Anshul Sharma

Designation: AI Engineer

Organization: IDS Infotech Ltd.

Date: _____

Signature of Student

Name: Hardik Sharma

Enrolment No: 2353777736

Date: _____

Signature of Coordinator / Study Centre

Date: _____

DECLARATION

I, **Hardik Sharma**, Enrolment No. **2353777736**, a student of Master of Computer Applications (MCA) at Indira Gandhi National Open University, hereby declare that the project work presented in this report entitled “**AI-Based Real-Time eKYC System for Deepfake-Free Identity Verification**” has been carried out by me under the supervision of Mr. Anshul Sharma, AI Engineer, IDS Infotech Ltd., Chandigarh.

I declare that:

1. This work is entirely my own and has not been submitted to any other university or institution for the award of any degree or diploma.
2. All information, data, facts and figures presented in this report are correct to the best of my knowledge.
3. I have acknowledged all sources of information and have cited them appropriately wherever they have been used.
4. I understand that any falsification or plagiarism is a serious academic offence and may result in cancellation of the degree.

Hardik Sharma

Enrolment No: 2353777736

MCA (MCAOL), IGNOU

Date: _____

ACKNOWLEDGEMENTS

I am grateful to all who contributed to the successful completion of this project.

I express my sincere gratitude to my project supervisor **Mr. Anshul Sharma**, AI Engineer at IDS Infotech Ltd., Chandigarh, for his invaluable guidance, constant encouragement, and constructive feedback throughout the development of this system. His expertise in artificial intelligence and practical industry insights were indispensable in shaping both the technical direction and the quality of this work.

I would like to thank the **School of Computer and Information Sciences, IGNOU**, for providing this learning opportunity through the MCSP-232 project course and for the structured guidelines that helped me plan and execute the project systematically.

I am deeply thankful to the open-source community – particularly the contributors to **FastAPI, React, SQLAlchemy, Chart.js, dlib, Hugging Face Transformers, and Google Generative AI** – whose tools form the backbone of this implementation.

Finally, I thank my family and friends for their constant moral support and encouragement throughout the programme.

Hardik Sharma

Enrolment No: 2353777736

ABSTRACT

Digital identity verification has become a cornerstone of modern financial services, with Know Your Customer (KYC) processes mandatory under Reserve Bank of India (RBI) guidelines. However, conventional eKYC systems that rely on static document uploads and simple selfie comparisons are increasingly vulnerable to sophisticated fraud techniques including GAN-generated synthetic faces, face-swap deepfakes, and morphed documents. This project presents the design and implementation of an **AI-Based Real-Time eKYC System** that addresses these vulnerabilities through a five-stage automated pipeline.

The system integrates **Google Gemini 2.5 Flash Lite** for multi-document OCR extraction (supporting Aadhaar, PAN, Passport, Driving Licence, and Voter ID), **dlib-based facial landmark detection** for isolating face regions from identity documents, a **Gradio-hosted face-similarity model** for biometric face matching with cosine-similarity scoring, a **HuggingFace deepfake detection model** for identifying AI-generated imagery, and a **Gradio-hosted liveness detection model** for anti-spoofing verification. An automated decision engine aggregates all scores against configurable thresholds to produce an approval or manual-review verdict, with a generated PDF report summarising all findings.

The backend is implemented in **Python 3.12 with FastAPI** and asynchronous SQLAlchemy 2.0, backed by SQLite (local development) or PostgreSQL (production), with optional MongoDB for audit logs and Redis for rate limiting. The frontend is built with **React 18 and TypeScript 5.6** using Vite, featuring a single-page scrollable pipeline interface, a webcam-capture selfie component, and an analytics dashboard with Chart.js visualisations for admin and reviewer roles. The system is containerised using **Docker Compose** and exposes a fully documented REST API.

Automated testing comprises 11 backend tests (pytest) and 10 frontend component tests (Vitest), all passing. The system demonstrates a practical, extensible architecture for real-time identity verification, providing a foundation that can be extended with mobile applications, blockchain audit trails, and government API integrations.

Keywords: eKYC, deepfake detection, face matching, liveness detection, OCR, FastAPI, React, identity verification, RBAC, JWT.

Contents

1	Introduction and Objectives	8
1.1	Background and Context	8
1.2	Motivation	9
1.3	Project Objectives	9
1.3.1	Primary Objective	9
1.3.2	Specific Objectives	9
1.4	Scope of the Project	11
1.4.1	In Scope	11
1.4.2	Out of Scope	11
1.5	Project Category and Technology Domain	12
1.6	Organisation of the Report	12
2	System Analysis and Need Identification	13
2.1	Study of Existing Systems	13
2.1.1	Commercial Solutions	13
2.1.2	Open-Source Tools	14
2.2	Problem Identification and Pain Points	14
2.3	Requirement Gathering Methodology	15
2.4	Feasibility Analysis	15
2.4.1	Technical Feasibility	15
2.4.2	Economic Feasibility	16
2.4.3	Operational Feasibility	16
2.4.4	Legal Feasibility	17
2.5	System Overview	17
3	Project Planning and Scheduling	19
3.1	Work Breakdown Structure	19

3.2	Project Milestones	20
3.3	Gantt Chart	20
3.4	PERT Chart	21
3.5	Team Roles and Responsibilities	23
3.6	Risk Assessment and Mitigation	24
3.7	Timeline Deviations and Lessons Learned	24
4	Software Requirements Specification	26
4.1	Introduction	26
4.1.1	Purpose	26
4.1.2	Document Conventions	26
4.1.3	Intended Audience	26
4.1.4	System Scope	26
4.2	Overall Description	27
4.2.1	Product Perspective	27
4.2.2	User Classes	27
4.2.3	Design and Implementation Constraints	27
4.2.4	Assumptions and Dependencies	28
4.3	Functional Requirements	28
4.4	Non-Functional Requirements	31
4.4.1	Performance Requirements (NFR-01)	31
4.4.2	Security Requirements (NFR-02)	31
4.4.3	Reliability Requirements (NFR-03)	31
4.4.4	Maintainability Requirements (NFR-04)	32
4.4.5	Portability Requirements (NFR-05)	32
4.4.6	Scalability Requirements (NFR-06)	32
4.5	External Interface Requirements	32
4.5.1	User Interface	32
4.5.2	Hardware Interface	33
4.5.3	Software Interfaces	33
4.6	Acceptance Criteria	33
5	Engineering Paradigm and Methodology	34
5.1	SDLC Approach	34
5.1.1	Iterative and Incremental Development	34

5.2	Modular Architecture Philosophy	35
5.3	API-First Design	36
5.4	Test-Driven Mindset	36
5.5	Traceability Matrix	36
6	System Modeling and Diagrams	37
6.1	Data Flow Diagrams	37
6.1.1	DFD Level 0 – Context Diagram	37
6.1.2	DFD Level 1	38
6.2	Use Case Diagram	42
6.3	Activity Diagram – Verification Workflow	45
6.4	Sequence Diagrams	48
6.4.1	User Verification Sequence	48
6.4.2	Admin Review Sequence	49
6.5	State Machine Diagram – Verification Session	50
6.6	Class Diagram – Backend Domain Model	51
6.7	ER Diagram	51
6.8	Deployment Diagram	52
6.9	Decision Control Flow Diagram	55
7	System Design	57
7.1	Modular Design Overview	57
7.2	Backend Architecture	58
7.2.1	Directory Structure	58
7.2.2	Layered Architecture	59
7.2.3	Feature Pipeline Route Design	59
7.3	Database Design	60
7.3.1	Schema Definition	60
7.3.2	Normalisation Analysis	62
7.4	API Design Principles	62
7.4.1	REST Conventions	62
7.4.2	Authentication Flow	63
7.5	Frontend Architecture	63
7.5.1	Component Hierarchy	63
7.5.2	State Management	64

7.6	UI Design	64
7.7	Error Handling Strategy	64
8	Coding Standards and Implementation	66
8.1	Coding Standards	66
8.1.1	Backend (Python)	66
8.1.2	Frontend (TypeScript)	66
8.2	Key Algorithms and Implementation Details	67
8.2.1	Document OCR with Google Gemini	67
8.2.2	Face Extraction from Document using dlib	68
8.2.3	Face Similarity via Gradio	69
8.2.4	Deepfake Detection via HuggingFace Inference API	70
8.2.5	Decision Engine	70
8.2.6	Webcam Selfie Capture (Frontend)	71
8.3	Parameter Passing and Data Flow	72
8.3.1	Backend Parameter Passing	72
8.3.2	Frontend Data Flow	73
9	Testing and Debugging	74
9.1	Testing Strategy	74
9.2	Backend Tests (pytest)	74
9.2.1	Test Configuration	74
9.2.2	Test Results	74
9.2.3	Selected Backend Test Implementations	75
9.3	Frontend Tests (Vitest)	76
9.3.1	Test Configuration	76
9.3.2	Test Results	76
9.3.3	Sample Frontend Test	77
9.4	System-Level Test Cases	77
9.5	Negative Test Cases	79
9.6	Debugging Log and Fixes	80
10	Security, Access Rights, and Data Protection	81
10.1	Authentication Mechanism	81
10.2	Password Security	82
10.3	Role-Based Access Control (RBAC)	82

10.4 Rate Limiting	83
10.5 Input Validation and Sanitisation	83
10.6 CORS Configuration	84
10.7 Audit Logging	84
10.8 Security Controls Summary	86
10.9 Known Security Limitations	86
11 Results, Reports, and Screens	88
11.1 System Interface – Login and Registration	88
11.2 Stage 1 – Document OCR Results	88
11.3 Stage 2 – Face Verification Results	89
11.4 Stage 3 – Deepfake Detection Results	90
11.5 Stage 4 – Liveness Detection Results	90
11.6 Stage 5 – Final Decision and PDF Report	90
11.7 Analytics Dashboard (Admin/Reviewer View)	91
11.7.1 KPI Cards	91
11.7.2 Charts	92
11.8 Admin Review Queue	92
12 Conclusion and Future Scope	93
12.1 Achievements	93
12.2 Limitations	94
12.3 Future Scope	95
12.3.1 Short-Term Enhancements (0-6 months)	95
12.3.2 Medium-Term Enhancements (6-18 months)	95
12.3.3 Long-Term Enhancements (18+ months)	95
12.4 Concluding Remarks	96
Bibliography	97
Glossary	99
A API Endpoint Catalog	102
B Database Schema Summary	104
C Technologies and Library Versions	106

List of Figures

2.1	System Architecture Overview showing frontend, backend, databases, and external services	17
3.1	Gantt Chart showing the 24-week project timeline with overlapping phases	20
3.2	PERT Chart showing task dependencies and critical path	22
6.1	DFD Level 0 – System Context Diagram	38
6.2	DFD Level 1 – Major System Processes	39
6.3	Use Case Diagram showing actor-system interactions	42
6.4	Activity Diagram showing the full user verification workflow	45
6.5	Sequence Diagram – User Identity Verification Flow	48
6.6	Sequence Diagram – Admin Review Workflow	49
6.7	State Machine Diagram for a Verification Session	50
6.8	Class Diagram – Backend Domain Model	51
6.9	Entity-Relationship Diagram	51
6.10	Deployment Diagram showing Docker Compose service topology	52
6.11	Decision Engine Control Flow	55
7.1	System Module Architecture	57

List of Tables

2.1	Open-Source Tools for Identity Verification	14
3.1	Work Breakdown Structure	19
3.2	Key Project Milestones	20
3.3	Team Roles and Responsibilities	23
3.4	Risk Register	24
4.1	User Classes and Characteristics	27
4.2	Functional Requirements	28
4.3	External Software Interfaces	33
5.1	Sprint Plan	35
5.2	Requirements Traceability Matrix (Sample)	36
7.1	Feature Pipeline API Endpoints	60
9.1	Backend Test Results	75
9.2	Frontend Test Results	77
9.3	System Test Case Matrix	78
9.4	Negative and Boundary Test Cases	79
9.5	Significant Bugs Identified and Fixed	80
10.1	RBAC Permission Matrix	82
10.2	Security Controls Implementation Status	86
11.1	Typical Stage 1 OCR Output	89
11.2	Sample Final Report Summary	91
A.1	Complete REST API Endpoint Reference	102
B.1	users Table Schema	104

B.2	verification_sessions Table Schema	105
C.1	Backend Technology Stack	106
C.2	Frontend Technology Stack	107
C.3	Infrastructure and Deployment Stack	107

Chapter 1

Introduction and Objectives

1.1 Background and Context

The digital revolution in financial services has fundamentally changed the way institutions onboard and verify customers. Electronic Know Your Customer (eKYC) processes allow organisations to verify a customer's identity remotely, replacing the cumbersome physical document submission model with a fully digital workflow. In India, the Reserve Bank of India (RBI) has made digital KYC mandatory for banks, non-banking financial companies (NBFCs), and payment service providers under its Master Direction on KYC (2016, updated 2023), specifying requirements for Video Customer Identification Process (V-CIP), liveness detection, and tamper-proof audit trails.

Despite the convenience of digital onboarding, eKYC fraud has escalated dramatically. The global financial sector reported a **300% increase** in deepfake-related fraud attempts between 2020 and 2023. Attackers now routinely use:

- **GAN-generated synthetic faces:** Photo-realistic images of non-existent people created by Generative Adversarial Networks (GANs).
- **Face-swap deepfakes:** Videos or images in which an attacker's face is digitally replaced by a victim's face.
- **Morphed documents:** Identity documents in which biometric photographs have been subtly altered to match an impersonator.
- **Photo/video replay attacks:** Presenting printed photographs or recorded videos to fool liveness detection cameras.

Existing eKYC systems – including some commercially deployed ones – rely on static image similarity scores and basic motion-detection liveness checks that fail against these advanced attacks. There is a clear need for a **multi-layered, AI-driven verification pipeline** that can simultaneously process document authenticity, biometric face matching, deepfake detection, and liveness verification in near real-time.

1.2 Motivation

The primary motivation for this project is threefold:

(1) Regulatory compliance: RBI mandates robust V-CIP with liveness detection for new-age financial services. Fintech startups and NBFCs face penalties for non-compliant KYC processes.

(2) Fraud prevention: With deepfake generation becoming accessible through freely available tools and pre-trained models, financial institutions are at increasing risk of synthetic identity fraud. A detection layer that screens every submitted selfie against deepfake indicators adds a critical security barrier.

(3) Academic and technical exploration: The project provides an opportunity to integrate multiple state-of-the-art AI services – large vision-language models for OCR, pre-trained facial recognition pipelines, and neural deepfake detectors – within a production-grade software architecture, demonstrating how complex AI capabilities can be composed into real-world applications.

1.3 Project Objectives

1.3.1 Primary Objective

Design, implement, and demonstrate a comprehensive AI-powered eKYC platform that performs multi-stage identity verification including document OCR, face matching, deepfake detection, and liveness analysis, producing a verifiable audit-ready PDF report.

1.3.2 Specific Objectives

- 1. Document OCR and Validation:** Extract identity fields (name, date of birth, document number, address) from Aadhaar, PAN, Passport, Driving Licence, and

- Voter ID using a Vision-Language Model (VLM), validate extracted fields using document-type-specific regex patterns, and assess document quality.
2. **Face Extraction:** Automatically detect and crop the facial photograph embedded within a submitted identity document using dlib's 68-point facial landmark detector, producing a reference face image for subsequent matching.
 3. **Biometric Face Matching:** Compare the reference face from the document against a live selfie image (uploaded or webcam-captured) using an external face-similarity model, returning a percentage match score with a pass/fail verdict.
 4. **Deepfake Detection:** Screen every submitted selfie through a pre-trained AI-image detector hosted on HuggingFace Inference API, returning a confidence score indicating the probability that the image is AI-generated or manipulated.
 5. **Liveness Detection:** Verify that the submitted selfie originates from a live person rather than a static photograph or replay attack using an external liveness model.
 6. **Automated Decision Engine:** Aggregate document authenticity, face match, deepfake, and liveness scores against configurable thresholds to produce an automated approved or manual_review decision, along with a composite final score.
 7. **PDF Report Generation:** Generate a downloadable PDF verification report containing all extracted data, individual stage scores, the final decision, and a unique verification identifier.
 8. **Admin Review Workflow:** Provide reviewer and admin roles with a flagged-session queue, enabling manual review with approval, rejection, or re-upload decisions, and propagating the decision to the user's KYC status.
 9. **Analytics Dashboard:** Provide admin-only analytics visualisations covering daily KYC volume trends, status distribution, daily approval rates, and pipeline stage latency.
 10. **Security and Compliance:** Implement JWT-based authentication, bcrypt password hashing, role-based access control (RBAC), rate limiting, input validation, and audit logging meeting industry security standards.

1.4 Scope of the Project

1.4.1 In Scope

- User registration and JWT-authenticated login (email, phone, or full name).
- Document upload and OCR processing for five document types.
- Webcam and file-upload selfie capture for face verification.
- Three AI inference stages: face match, deepfake detection, liveness.
- Automated decision engine with configurable scoring thresholds.
- PDF report generation and download.
- Admin/reviewer manual review workflow.
- Role-based analytics dashboard with Chart.js visualisations.
- Audit logging to MongoDB (optional, gracefully degraded if unavailable).
- Docker Compose full-stack deployment.
- REST API documented via FastAPI's built-in OpenAPI/Swagger interface.

1.4.2 Out of Scope

- Native iOS/Android mobile applications.
- Real-time video streaming verification (video liveness is limited to image-based checks in the current implementation).
- Direct integration with UIDAI Aadhaar API, DigiLocker, or other government databases.
- Multi-language OCR support beyond English.
- Fingerprint or iris biometric modalities.
- Payment gateway integration.
- Production-grade email/SMS delivery (currently mock-logged).
- Blockchain-based immutable audit trails.

1.5 Project Category and Technology Domain

Primary Category: Artificial Intelligence, Machine Learning, Deep Learning

Secondary Categories: Computer Vision, Image Processing, Web Development, RDBMS
+ NoSQL, Software Engineering, Information Security

1.6 Organisation of the Report

The remainder of this report is organised as follows:

- **Chapter 2** presents the system analysis, including a study of existing systems, pain points, and feasibility analysis.
- **Chapter 3** covers project planning with Gantt and PERT charts, Work Breakdown Structure, and risk assessment.
- **Chapter 4** provides the full Software Requirements Specification.
- **Chapter 5** describes the engineering paradigm and methodology.
- **Chapter 6** presents all system models and UML diagrams.
- **Chapter 7** details the system design including database and API design.
- **Chapter 8** covers implementation, coding standards, and key algorithms.
- **Chapter 9** presents testing strategies and results.
- **Chapter 10** discusses security mechanisms and access controls.
- **Chapter 11** shows representative system outputs and screenshots.
- **Chapter 12** concludes with achievements, limitations, and future scope.

Chapter 2

System Analysis and Need Identification

2.1 Study of Existing Systems

Several commercial and open-source eKYC solutions currently exist in the market. A survey of these systems reveals both their capabilities and their shortcomings, which motivate the approach taken in this project.

2.1.1 Commercial Solutions

DigiLocker-Linked KYC (India): DigiLocker provides government-backed document sharing where users authorise access to UIDAI-issued digital Aadhaar. While it eliminates physical document submission, it has no built-in liveness or deepfake detection, and requires users to have a DigiLocker account linked to their Aadhaar number.

IDFY (India): IDFY provides API-based OCR and face-match services. Its offerings are cloud-only, per-API-call charged, require vendor lock-in, and do not expose an open architecture for academic or custom integration.

Jumio / Onfido (Global): These are market-leading KYC-as-a-service platforms with sophisticated AI. However, they are proprietary, expensive (enterprise licensing), closed-source, and cannot be studied, modified, or deployed in an academic context.

Limitations of existing systems:

- Most commercial systems treat deepfake detection as an optional add-on, not as a

first-class verification stage.

- None provide a transparent, self-hosted, open-architecture pipeline where the verification logic can be inspected and customised.
- Academic institutions and startups cannot afford commercial APIs for research or proof-of-concept implementations.
- Audit logs in commercial systems are opaque; there is no mechanism to export or analyse raw verification data.

2.1.2 Open-Source Tools

Several open-source libraries address individual stages but no integrated open pipeline exists:

Table 2.1: Open-Source Tools for Identity Verification

Tool	Purpose	Limitation
Tesseract OCR	Text extraction	No field understanding, poor on ID layouts
InsightFace	Face recognition	Model only; no pipeline integration
dlib	Face detection	Low-level; requires custom integration
FaceNet	Face embeddings	Training required; no turnkey service
Various deepfake detectors	Image classification	Standalone models; no API wrapper

2.2 Problem Identification and Pain Points

Based on the study of existing systems, the following key problems were identified:

1. **No integrated pipeline:** No single open-source system combines OCR, face extraction, face matching, deepfake detection, and liveness in a unified REST API.
2. **Deepfake blindness:** Existing simple eKYC tools do not screen selfies for AI generation, leaving a critical security gap.
3. **Static document images:** OCR alone cannot determine whether a document image is authentic or digitally tampered.
4. **Poor admin tooling:** No open platform provides a review queue where flagged sessions can be manually adjudicated with a structured audit trail.

5. **No analytics:** Verification trends, stage-wise latency, and approval rates are not visible in typical open-source setups.
6. **Lack of transparency:** Commercial systems give a binary pass/fail; there is no per-stage score breakdown that allows reviewers to understand the basis of a decision.

2.3 Requirement Gathering Methodology

Requirements for this project were gathered through multiple structured approaches:

(a) **Regulatory Document Review:** The RBI Master Direction on KYC (2023) was reviewed to identify mandatory functional requirements for eKYC systems, particularly V-CIP requirements.

(b) **Literature Survey:** Academic papers on face recognition (ArcFace, FaceNet), deep-fake detection (dual-stream architectures, frequency-domain analysis), and liveness detection (texture-based, depth-based) were reviewed to identify state-of-the-art approaches.

(c) **Competitive Analysis:** Features of Jumio, Onfido, and IDfy were catalogued to establish a feature-parity baseline.

(d) **User Persona Analysis:** Three distinct user personas were defined – end user (verifies identity), reviewer (reviews flagged cases), and admin (system oversight + analytics) – and their workflows were traced to derive functional requirements.

(e) **Prototyping:** An initial prototype of the five-stage pipeline was built and tested with real documents to identify edge cases, error conditions, and latency bottlenecks.

2.4 Feasibility Analysis

2.4.1 Technical Feasibility

All required technologies are mature, well-documented, and available:

- **FastAPI** (Python 3.12) provides high-performance async REST API development with built-in OpenAPI documentation.
- **Google Gemini 2.5 Flash Lite** provides state-of-the-art Vision-Language OCR capability via a simple API.

- **dlib** (C++ with Python bindings) provides robust facial landmark detection for face extraction from documents.
- **HuggingFace Inference API** provides hosted deepfake detection without requiring local GPU resources.
- **Gradio Spaces** provide hosted face-similarity and liveness models accessible over HTTP.
- **React + TypeScript** provides a modern, type-safe frontend framework with an extensive ecosystem.
- **SQLAlchemy 2.0 async** provides production-grade ORM with PostgreSQL and SQLite support.
- **Docker Compose** enables reproducible multi-service deployment.

2.4.2 Economic Feasibility

- All core frameworks and libraries are open-source (zero licensing cost).
- Gemini API, HuggingFace Inference API, and Gradio Spaces all offer free tiers sufficient for development and demonstration purposes.
- Infrastructure cost for local development is zero; Docker-based deployment on commodity hardware is feasible.
- Production deployment on a cloud provider (AWS EC2 t3.medium) would cost approximately \$40–\$80/month – viable for a startup.

2.4.3 Operational Feasibility

- The system’s web-based interface requires no software installation on the user side beyond a modern browser.
- Admin and reviewer workflows are simple (list → review → decide) and require minimal training.
- Automated score-based decisions reduce manual review workload to only borderline cases.
- The Docker Compose setup makes deployment and operations straightforward for a small DevOps team.

2.4.4 Legal Feasibility

- All external APIs and models used are available under terms permitting research and academic use.
- The system's architecture is designed with GDPR/PDPB-compatible data practices: document images are stored per-session, audit logs are retained, and user data deletion is supported via cascade-delete foreign keys.
- The system does not directly integrate with UIDAI or NPCI systems, avoiding regulatory requirements associated with licensed API consumers.

2.5 System Overview

The system can be described at a high level as a five-stage sequential pipeline with a supporting administration layer:

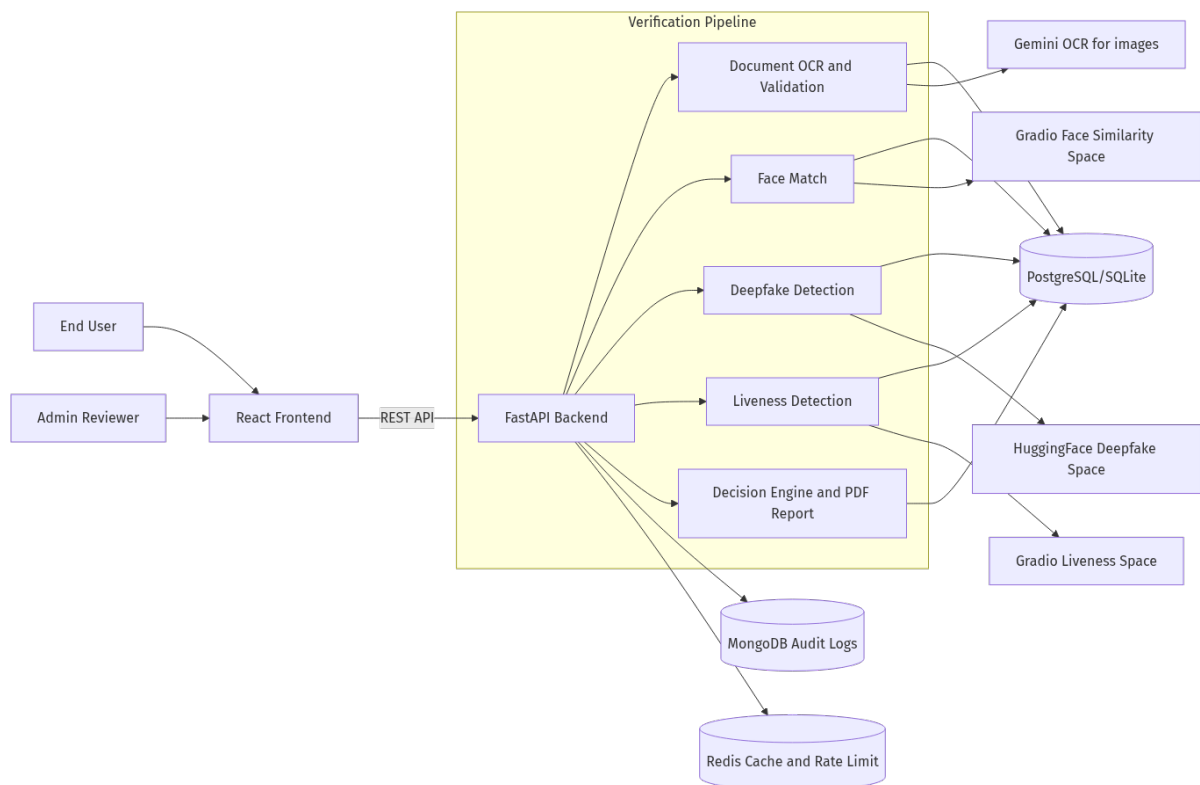


Figure 2.1: System Architecture Overview showing frontend, backend, databases, and external services

1. **Stage 1 – Document OCR:** User uploads an identity document. The backend

sends the image to Google Gemini for structured field extraction, validates the results, extracts a face crop using dlib, and stores both the OCR data and image references.

2. **Stage 2 – Face Matching:** User provides a live selfie (file upload or webcam). The backend compares the document face crop against the selfie using an external face-similarity Gradio service and returns a percentage score.
3. **Stage 3 – Deepfake Detection:** The backend submits the selfie image to a HuggingFace-hosted AI image detector and returns a deepfake probability score.
4. **Stage 4 – Liveness Detection:** The backend submits the selfie to a Gradio-hosted liveness model and returns a liveness score.
5. **Stage 5 – Final Decision + Report:** An automated decision engine aggregates all four scores. If all thresholds are met, the session is approved; otherwise it is flagged for manual review. A PDF report is generated and downloadable.

The administration layer provides a reviewer queue for flagged sessions and an analytics dashboard visible to admin and reviewer roles.

Chapter 3

Project Planning and Scheduling

3.1 Work Breakdown Structure

The project was decomposed into the following work packages:

Table 3.1: Work Breakdown Structure

WBS ID	Work Package	Sub-Tasks	Deliverable
1.0	Requirements	Literature review, SRS draft	SRS document
2.0	Architecture	DB design, API design, module breakdown	Design doc
3.0	Backend Core	Auth, DB models, migrations	Working API
4.0	Document Module	Upload, OCR, validation, face extract	OCR endpoint
5.0	Biometric Module	Face match, deepfake, liveness	3 AI endpoints
6.0	Decision + Report	Scoring engine, PDF generation	Report endpoint
7.0	Admin Module	Review queue, decision API, analytics	Admin routes
8.0	Frontend	Auth UI, 5 pipeline sections, dashboard	React app
9.0	Integration	End-to-end pipeline testing	Working system
10.0	Docker	Compose setup, environment config	docker-compose.yml
11.0	Testing	Unit, integration, system tests	Test suite
12.0	Documentation	Report, API docs, code comments	This report

3.2 Project Milestones

Table 3.2: Key Project Milestones

No.	Milestone	Target Week	Status
M1	SRS approved	Week 3	Completed
M2	Database schema finalised	Week 5	Completed
M3	Auth module operational	Week 7	Completed
M4	Document OCR pipeline working	Week 10	Completed
M5	Face match + deepfake + liveness operational	Week 14	Completed
M6	Decision engine + PDF report working	Week 16	Completed
M7	Admin review queue working	Week 17	Completed
M8	Frontend all sections integrated	Week 20	Completed
M9	Analytics dashboard with charts	Week 21	Completed
M10	Docker Compose full stack working	Week 22	Completed
M11	All automated tests passing	Week 23	Completed
M12	Report submitted	Week 24	Completed

3.3 Gantt Chart

The project followed a 24-week schedule with overlapping phases to allow parallel development:

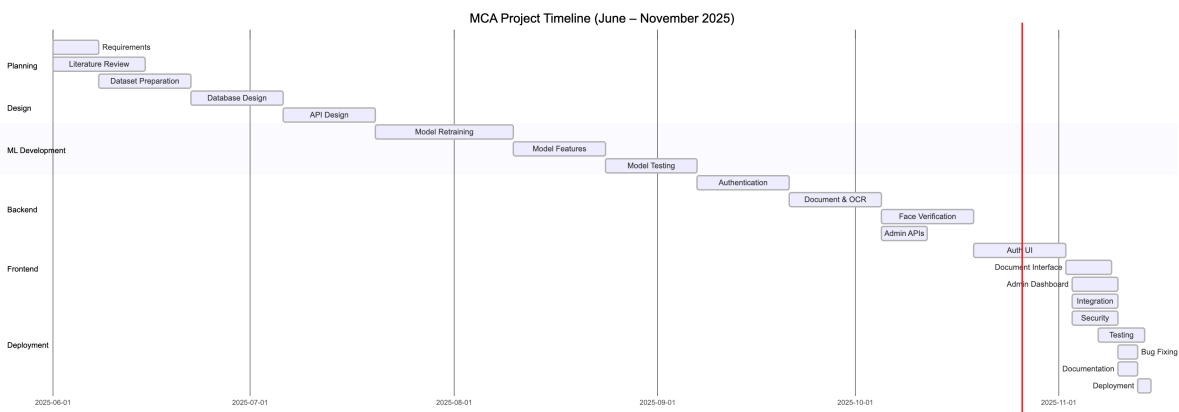


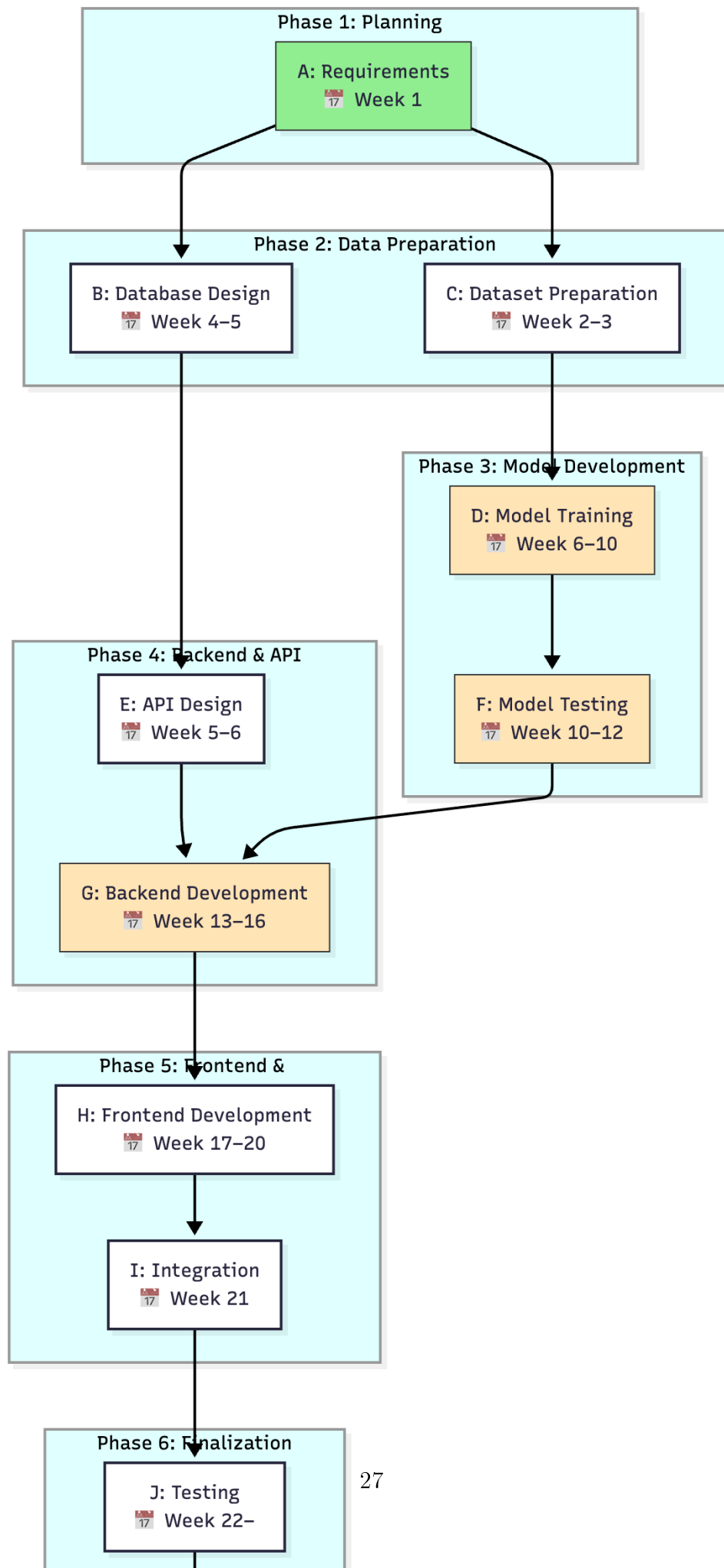
Figure 3.1: Gantt Chart showing the 24-week project timeline with overlapping phases

The overlapping design allowed backend and frontend development to proceed in parallel

once the API contracts were established at Week 5, significantly reducing the overall project duration compared to a purely sequential waterfall approach.

3.4 PERT Chart

The Programme Evaluation and Review Technique (PERT) chart below shows the critical path through the project:



Critical Path:

Requirements Analysis → Database Design → Backend Core → Document OCR → Face Match → Deepfake Detection → Liveness → Decision Engine → PDF Report → Admin Module → Integration Testing → Submission

Total critical path duration: 24 weeks.

3.5 Team Roles and Responsibilities

Table 3.3: Team Roles and Responsibilities

Role	Person	Responsibilities
Developer / Architect	Hardik Sharma	Full-stack implementation, all modules
Supervisor	Anshul Sharma	Technical review, guidance, approval

3.6 Risk Assessment and Mitigation

Table 3.4: Risk Register

Risk		Probability	Impact	Mitigation
External API unavailability (Gradio, Hugging-Face)		Medium	High	Graceful error handling; mock responses for testing
Gemini OCR in-accuracy on low-quality documents		High	Medium	Pre-processing, quality scoring, user feedback
SQLite limitations in production		Low	Medium	Docker uses PostgreSQL; SQLite only for local dev
MongoDB unavailability		Low	Low	Audit logging degrades silently; app continues
JWT secret compromise		Low	Very High	Env var only; no hard-coding; documented in deployment notes
Scope creep		Medium	Medium	Strict in-scope/out-of-scope definition; deferred features listed

3.7 Timeline Deviations and Lessons Learned

- **OCR field extraction (Week 8-10):** Initial use of Tesseract OCR produced poor results on Indian identity documents due to varied fonts and layouts. This was replaced with Google Gemini 2.5 Flash Lite, which significantly improved extraction accuracy but required additional API integration work (+1 week).
- **dlib installation (Week 9):** Compiling dlib from source on macOS required resolving cmake and boost dependencies (+2 days).
- **Frontend webcam integration (Week 20):** Browser-based webcam capture using getUserMedia API required additional state management and a hidden canvas element for snapshot capture (+1 day).

- **Overall:** The project was delivered within the planned 24-week window, with no critical path delays.

Chapter 4

Software Requirements Specification

4.1 Introduction

4.1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for the AI-Based Real-Time eKYC System. It serves as the contractual agreement between the developer and the supervisor, and as the baseline against which the implemented system is validated.

4.1.2 Document Conventions

- Requirements are tagged **FR-nn** (functional) or **NFR-nn** (non-functional).
- Priority is indicated as: **Must Have**, **Should Have**, or **Nice to Have**.
- Implementation status is: **Implemented**, **Partial**, or **Planned**.

4.1.3 Intended Audience

Developer, project supervisor, IGNOU evaluators.

4.1.4 System Scope

The system is a web-based eKYC platform consuming AI services for identity verification. It is not a government KYC system; it is an academic demonstration of an integrated AI pipeline for identity verification.

4.2 Overall Description

4.2.1 Product Perspective

The system is a standalone web application. It integrates with three categories of external services:

1. **Google Generative AI** (Gemini 2.5 Flash Lite) for document OCR.
2. **HuggingFace Inference API** for deepfake detection.
3. **Gradio Spaces** for face similarity and liveness detection.

4.2.2 User Classes

Table 4.1: User Classes and Characteristics

User Class	Description	Primary Tasks
End User	General public seeking KYC verification	Register, upload document, provide selfie, view status, download report
Reviewer	Staff with KYC review authority	View flagged sessions, approve/reject/request re-upload
Admin	System administrator	All reviewer actions + view analytics, audit logs

4.2.3 Design and Implementation Constraints

- OCR accuracy depends on input document image quality.
- Face match and liveness scores depend on availability and performance of external Gradio-hosted models.
- Deepfake detection depends on HuggingFace Inference API availability.
- Local development uses SQLite; full production stack uses PostgreSQL.
- MongoDB and Redis are optional; the system degrades gracefully if unavailable.

4.2.4 Assumptions and Dependencies

- Users have a device with a modern web browser (Chrome 90+, Firefox 88+, Safari 14+).
- Users have a camera for webcam-based selfie capture.
- `GEMINI_API_KEY`, `DEEPFAKE_HF_TOKEN`, `FACE_SIMILARITY_SPACE_URL`, and `LIVENESS_SPACE_URL` are configured in the backend environment.

4.3 Functional Requirements

Table 4.2: Functional Requirements

ID	Requirement	Priority	Status	Test ID
FR-01	User registers with full_name, email, phone, password; system validates uniqueness and returns JWT token	Must Have	Implemented	AUTH-01/02
FR-02	User logs in using email, phone, or full name plus password; system returns JWT token	Must Have	Implemented	AUTH-03/04
FR-03	Authenticated user can retrieve own profile (role, KYC status) via /auth/me	Must Have	Implemented	AUTH-05
FR-04	User can upload a document image (Aadhaar/-PAN/Passport/DL/Voter ID) as JPEG, PNG, or PDF	Must Have	Implemented	DOC-01
FR-05	System extracts OCR fields (name, DOB, document number, address) using Gemini Vision API	Must Have	Implemented	DOC-04

Table 4.2 – continued

ID	Requirement	Priority	Status	Test ID
FR-06	System validates document number format against type-specific regex patterns	Must Have	Implemented	DOC-04
FR-07	System calculates a document quality score and returns a recommendation	Should Have	Implemented	DOC-04
FR-08	System extracts and stores a face crop from the document image using dlib	Must Have	Implemented	DOC-04
FR-09	User provides a selfie image (file upload or webcam capture); system runs face similarity comparison	Must Have	Implemented	BIO-02
FR-10	Face match endpoint returns a percentage score and pass/fail result	Must Have	Implemented	BIO-02
FR-11	System screens the selfie through a deepfake detection model and returns a risk score	Must Have	Implemented	BIO-03
FR-12	System runs liveness detection on the selfie and returns a liveness score	Must Have	Implemented	BIO-04
FR-13	Decision engine aggregates all four scores and determines approved or manual_review	Must Have	Implemented	DEC-03
FR-14	System generates a downloadable PDF verification report after final decision	Must Have	Implemented	DEC-03

Table 4.2 – continued

ID	Requirement	Priority	Status	Test ID
FR-15	Reviewer/admin can list all flagged (manual_review) sessions	Must Have	Implemented	ADM-01
FR-16	Reviewer/admin can approve, reject, or request re-upload for a flagged session	Must Have	Implemented	ADM-04
FR-17	Rejection requires a reason string; system enforces this at the API level	Must Have	Implemented	ADM-03
FR-18	Admin can view analytics overview (total users, sessions, approvals, rejections, flagged)	Should Have	Implemented	ANA-01
FR-19	Admin can view 30-day daily trend data (approved/rejected/flagged per day)	Should Have	Implemented	ANA-02
FR-20	Admin can view per-stage average and P95 latency data	Should Have	Implemented	ANA-03
FR-21	System writes audit log entries to MongoDB for auth and review events	Should Have	Partial	AUD-01
FR-22	Authenticated image requests are proxied through a protected endpoint requiring a valid JWT	Must Have	Implemented	SEC-04

4.4 Non-Functional Requirements

4.4.1 Performance Requirements (NFR-01)

- API response time for all non-AI endpoints (auth, session CRUD, analytics): <500 ms at p95 under normal load.
- Document OCR stage: dependent on Gemini API; typically 3–10 s.
- Face match stage: dependent on Gradio Space; typically 2–5 s.
- Deepfake detection: dependent on HuggingFace Inference API; typically 5–15 s.
- Liveness detection: dependent on Gradio Space; typically 2–5 s.
- PDF generation: <2 s for a typical report payload.

4.4.2 Security Requirements (NFR-02)

- All passwords stored as bcrypt hash with cost factor 12.
- JWT tokens signed with HS256; configurable expiry (default 24 h).
- All API routes requiring authentication return HTTP 401 if no valid token is presented.
- Admin and reviewer routes return HTTP 403 if the token holder's role is insufficient.
- File uploads are validated for MIME type, extension, and size before processing.
- Rate limiting applied to all auth routes using slowapi.
- Input strings are sanitised before storage.

4.4.3 Reliability Requirements (NFR-03)

- Application startup creates all required database tables automatically (`AUTO_CREATE_SCHEMA=true`).
- External service failures (Gradio, HuggingFace, MongoDB, Redis) are caught and mapped to appropriate HTTP error responses; the core application remains operational.
- A `/api/v1/system/health` endpoint returns the status of all dependent services.

4.4.4 Maintainability Requirements (NFR-04)

- Backend follows a layered architecture: `api/routes` → `services` → `models` ↔ `db/session`.
- Frontend components are TypeScript-typed with explicit prop interfaces.
- All API endpoints are documented via FastAPI's auto-generated OpenAPI schema.
- Database schema changes are managed via Alembic migrations.

4.4.5 Portability Requirements (NFR-05)

- The full stack can be started with a single `docker compose up -d --build` command.
- Local development (without Docker) requires only Python 3.12, Node.js 18+, and environment variable configuration.
- The system supports both SQLite (local) and PostgreSQL (Docker/production) without code changes.

4.4.6 Scalability Requirements (NFR-06)

- FastAPI's async architecture enables handling multiple concurrent requests without thread blocking.
- Redis is integrated as a rate-limit store and cache, allowing horizontal scaling of backend instances.
- PostgreSQL supports connection pooling and read replicas for production scale-out.

4.5 External Interface Requirements

4.5.1 User Interface

- Single-page web application accessible via standard desktop and tablet browsers.
- Responsive layout with a sticky header, 5-step pipeline progress indicator, and section-based scrollable content.
- Toast notifications for all operations (success, warning, error).
- Collapsible raw JSON output for debugging.

4.5.2 Hardware Interface

- Server: any x86_64 machine with 4+ GB RAM and Python 3.12.
- Client: any device with a modern browser and an optional camera for webcam selfie capture.

4.5.3 Software Interfaces

Table 4.3: External Software Interfaces

System	Protocol	Config Var	Purpose
Google Gemini API	HTTPS / REST	GEMINI_API_KEY	Document OCR
HuggingFace Inference	HTTPS / REST	DEEPFAKE_HF_TOKEN	Deepfake scoring
Gradio Face Similarity	HTTPS / SSE	FACE_SIMILARITY_SPACE_URL	Face matching
Gradio Liveness	HTTPS / SSE	LIVENESS_SPACE_URL	Liveness scoring
PostgreSQL / SQLite	TCP / file	DATABASE_URL	Persistent data
MongoDB	TCP	MONGODB_URI	Audit logs
Redis	TCP	REDIS_URL	Rate limiting, cache

4.6 Acceptance Criteria

The following acceptance criteria define the minimum viable delivered system:

1. A new user can register, login, and receive a JWT token. (*AUTH-01*)
2. A user can upload a PAN/Aadhaar document and receive OCR-extracted fields. (*DOC-04*)
3. A user can submit a selfie and receive a face match score. (*BIO-02*)
4. A user can receive deepfake and liveness scores for a submitted selfie. (*BIO-03*, *BIO-04*)
5. A user can generate and download a PDF report after completing all pipeline stages. (*DEC-03*)
6. An admin can view the flagged session queue and approve a session. (*ADM-04*)
7. Analytics endpoints return valid aggregate data for admin tokens. (*ANA-01*)
8. All 11 backend tests and 10 frontend tests pass. (*Automated test suite*)

Chapter 5

Engineering Paradigm and Methodology

5.1 SDLC Approach

5.1.1 Iterative and Incremental Development

The project followed an **iterative and incremental** approach inspired by Agile principles, rather than a strict waterfall model. This choice was motivated by:

- **Evolving requirements:** The exact API contracts for external services (Gradio, HuggingFace) were discovered during implementation and required iterative adjustment.
- **Tight feedback loop:** Working software was available after each stage of the pipeline was implemented, allowing early validation.
- **Risk mitigation:** High-risk components (external AI services) were prototyped and integrated early, reducing late-stage surprises.

Development was organised into 2-week sprints, each with a specific deliverable:

Table 5.1: Sprint Plan

Sprint	Weeks	Focus	Deliverable
1	1–2	Requirements, architecture	SRS, DB schema
2	3–4	Backend core	Auth, DB models, health endpoint
3	5–6	Document module	Upload + OCR endpoint
4	7–8	Face extraction	dlib integration, face crop storage
5	9–10	Face match + deepfake	Gradio + HuggingFace integration
6	11–12	Liveness + decision	Liveness endpoint + scoring engine
7	13–14	PDF report	WeasyPrint generation + download
8	15–16	Admin module	Review queue + analytics endpoints
9	17–18	Frontend core	Auth UI + OCR + face match sections
10	19–20	Frontend pipeline	Deepfake, liveness, report sections
11	21–22	Frontend analytics + Docker	Chart.js dashboard + Compose setup
12	23–24	Testing + documentation	All tests passing, report written

5.2 Modular Architecture Philosophy

The system follows a **separation of concerns** principle with clearly defined module boundaries:

1. **Authentication Module:** Handles all identity-related concerns – registration, login, JWT issuance, token validation, and role resolution. No business logic from other modules bleeds into this layer.
2. **Feature Pipeline Module:** Implements the five-stage verification pipeline as a single coherent route group (`/api/v1/features/`). Each stage builds on the previous one through in-memory session state.
3. **Admin Module:** Provides review queue and decision APIs restricted to reviewer/admin roles. Decoupled from the feature pipeline.
4. **Analytics Module:** Provides aggregation queries over verification sessions. Read-only; does not mutate any data.
5. **Audit Module:** Writes to MongoDB. Implemented as a fire-and-forget async call that never blocks the main request flow.
6. **Frontend Components:** React components are organised by function (auth,

pipeline stages, dashboard, layout, shared). Each component receives data through typed props and calls API functions from the centralised `services/api.ts` module.

5.3 API-First Design

The backend was designed API-first: the OpenAPI schema was defined before frontend development began, ensuring a stable contract between the two layers. FastAPI auto-generates and serves this schema at `/docs` (Swagger UI) and `/redoc` (ReDoc).

All API functions on the frontend return a typed `{data, error}` object, isolating error handling from business logic.

5.4 Test-Driven Mindset

While the project did not follow strict Test-Driven Development (TDD), tests were written alongside implementation rather than after, particularly for:

- Auth routes (register, login, profile) – verifying security boundaries.
- Frontend components (LoginForm, ScoreCard, Toast) – verifying rendering and interaction.

5.5 Traceability Matrix

Table 5.2: Requirements Traceability Matrix (Sample)

FR ID	Module	File	Test ID
FR-01	Auth	<code>app/api/routes/auth.py</code>	AUTH-01
FR-02	Auth	<code>app/api/routes/auth.py</code>	AUTH-03
FR-05	Document	<code>app/services/document_processing.py</code>	DOC-04
FR-08	Document	<code>app/services/document_face_extraction.py</code>	DOC-04
FR-09	Feature	<code>app/api/routes/features.py</code>	BIO-02
FR-11	Feature	<code>app/services/deepfake_inference.py</code>	BIO-03
FR-13	Feature	<code>app/api/routes/features.py</code>	DEC-03
FR-18	Analytics	<code>app/api/routes/analytics.py</code>	ANA-01

Chapter 6

System Modeling and Diagrams

6.1 Data Flow Diagrams

6.1.1 DFD Level 0 – Context Diagram

The Level 0 DFD (Context Diagram) shows the eKYC system as a single process interacting with three external entities:

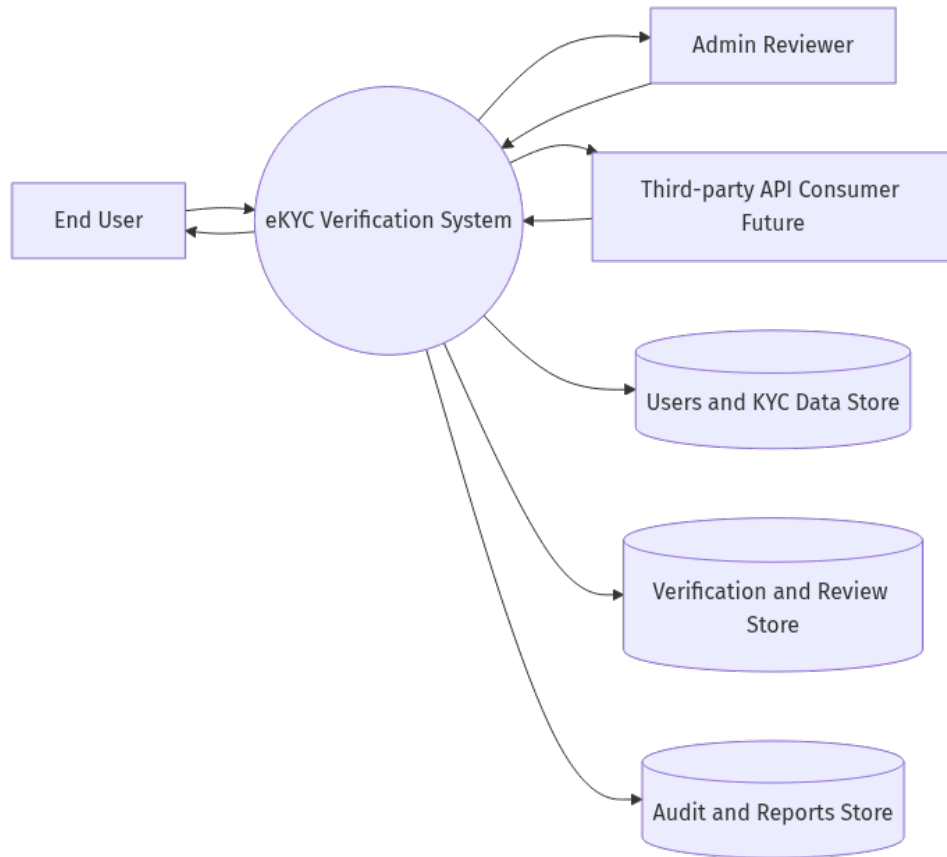


Figure 6.1: DFD Level 0 – System Context Diagram

External Entities:

- **End User:** Provides personal information, identity documents, and selfie images. Receives KYC status and PDF report.
- **Admin/Reviewer:** Receives flagged session data. Provides review decisions.
- **AI Services:** External platforms (Gemini, HuggingFace, Gradio) that receive document/image data and return analysis results.

6.1.2 DFD Level 1

The Level 1 DFD decomposes the system into seven major processes:

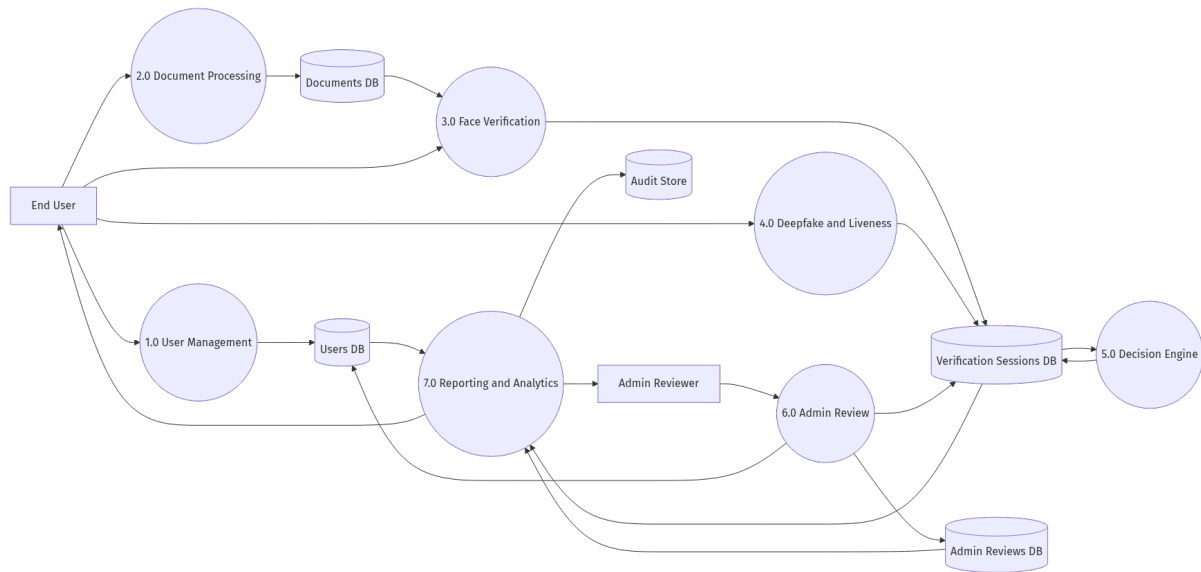


Figure 6.2: DFD Level 1 – Major System Processes

Process Descriptions:

1.0 – User Management: Handles registration, login, and profile management. Reads/writes the Users data store. Outputs JWT tokens to authenticated entities.

2.0 – Document Processing: Receives document images from the End User. Sends them to the OCR Engine (Gemini). Validates extracted data and invokes face extraction. Writes to the Documents data store.

3.0 – Face Verification: Receives the reference face crop (from Process 2.0) and the live selfie image. Invokes the Gradio face-similarity service. Writes match score and image references to the Sessions data store.

4.0 – Deepfake & Liveness Detection: Receives the live selfie image. Invokes the HuggingFace deepfake model and the Gradio liveness model. Writes scores to the Sessions data store.

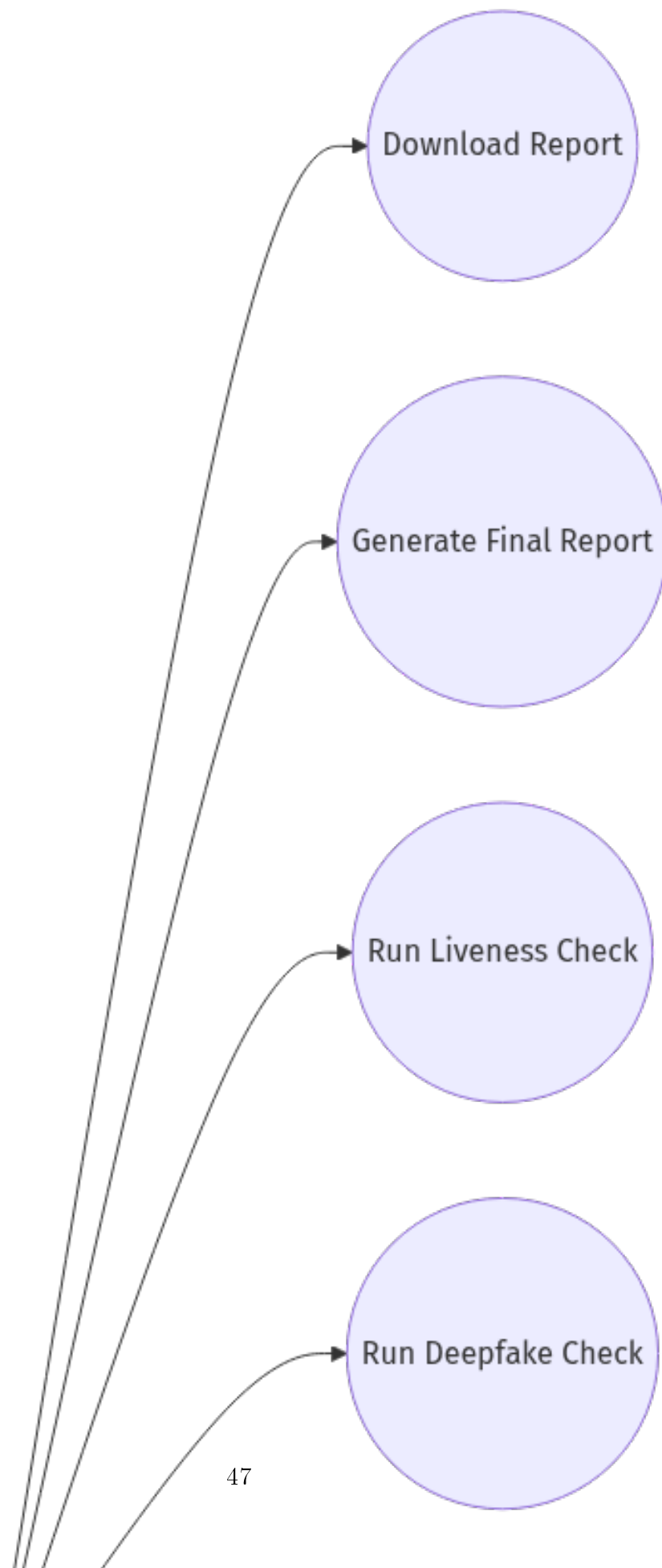
5.0 – Decision Engine: Reads all scores from the Sessions data store. Applies thresholds. Writes the final KYC decision to both the Sessions and Users data stores.

6.0 – Admin Review: Reads flagged sessions from the Sessions data store. Presents them to the Admin/Reviewer entity. Receives review decisions and updates the Reviews and Sessions data stores.

7.0 – Reporting: Reads session data and generates PDF reports. Writes to the Audit

Logs data store. Provides reports to the End User and analytics data to the Admin.

6.2 Use Case Diagram



Actor: End User

- UC-01: Register Account
- UC-02: Login
- UC-03: Upload Identity Document
- UC-04: Provide Selfie (Upload or Webcam)
- UC-05: View Verification Status
- UC-06: Download PDF Report
- UC-07: Logout

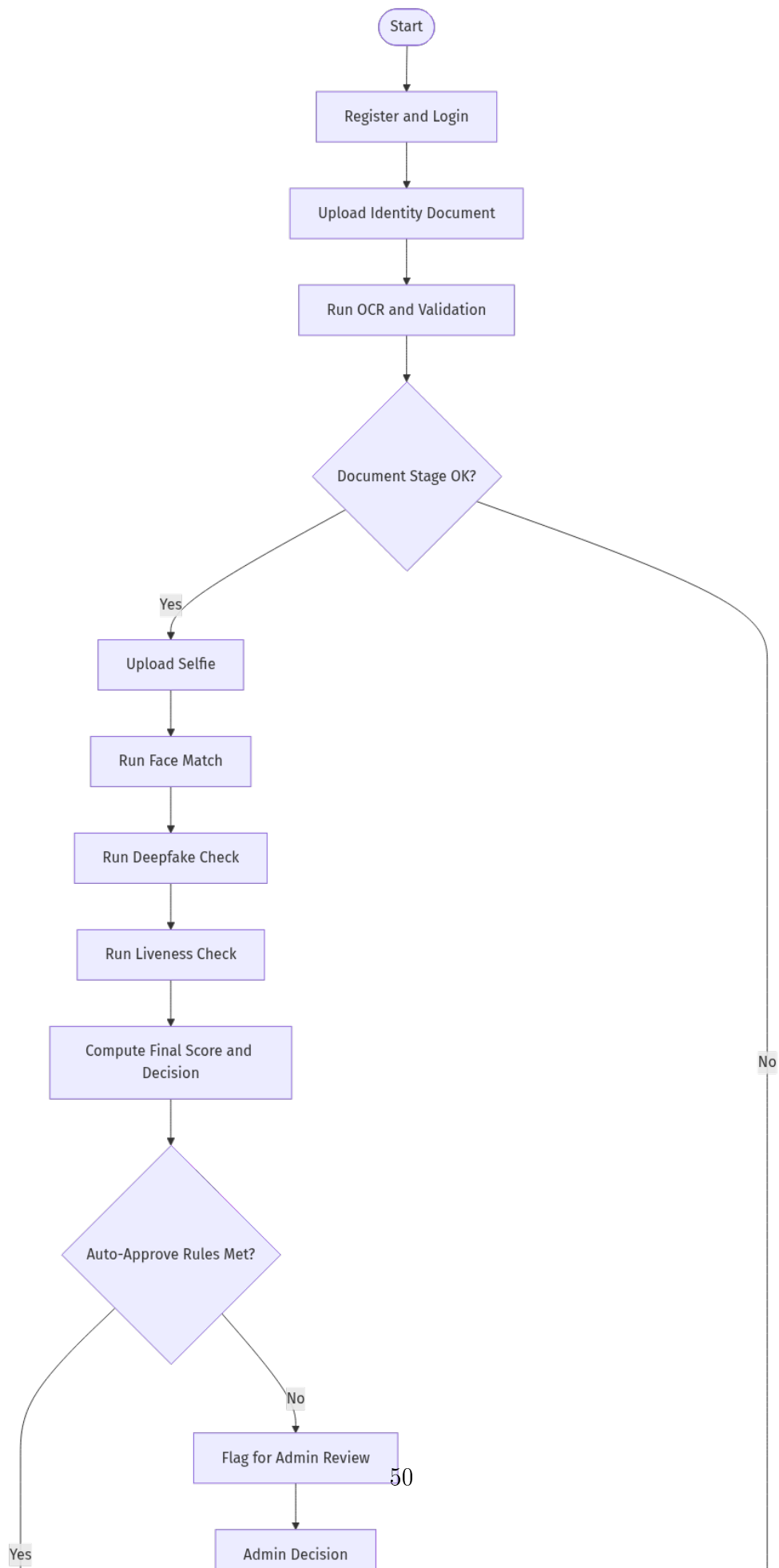
Actor: Reviewer (extends End User)

- UC-08: View Flagged Session Queue
- UC-09: Approve Flagged Session
- UC-10: Reject Flagged Session
- UC-11: Request Document Re-Upload

Actor: Admin (extends Reviewer)

- UC-12: View Analytics Dashboard
- UC-13: View Audit Logs
- UC-14: Manage Users

6.3 Activity Diagram – Verification Workflow

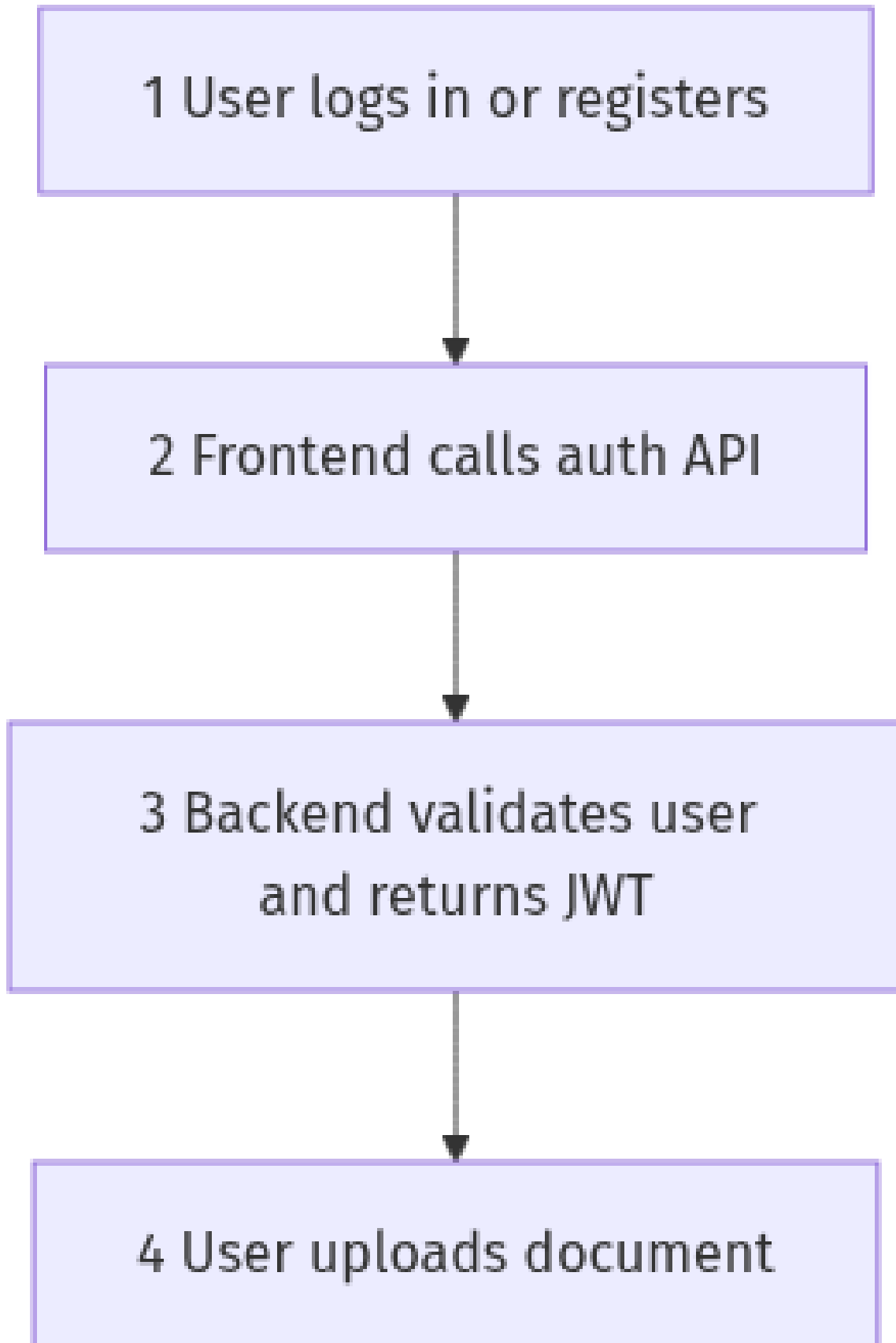


The verification workflow proceeds as follows:

1. User registers and logs in.
2. User uploads an identity document. System performs OCR extraction and face crop.
3. If OCR fails quality checks, user is prompted to re-upload.
4. User provides a live selfie (upload or webcam).
5. System runs face match, deepfake detection, and liveness in parallel.
6. Decision engine evaluates all scores.
7. **Branch A (All thresholds met):** Session is automatically approved. PDF report generated. User notified.
8. **Branch B (Any threshold failed):** Session flagged for manual review.
9. Admin reviews flagged session: approves, rejects, or requests re-upload.
10. User is notified of the final decision.

6.4 Sequence Diagrams

6.4.1 User Verification Sequence



The sequence shows interactions between: Browser, React SPA, FastAPI Backend, Gemini API, Gradio Face Service, HuggingFace API, and PostgreSQL Database.

Key interaction steps:

1. User POSTs credentials → Backend validates → Returns JWT.
2. User POSTs document image → Backend calls Gemini → Returns OCR data + face crop URL.
3. User POSTs selfie → Backend calls Gradio face-similarity → Returns match score.
4. Backend calls HuggingFace deepfake model → Returns deepfake probability.
5. Backend calls Gradio liveness model → Returns liveness score.
6. Backend runs decision engine → Persists decision → Returns final report.

6.4.2 Admin Review Sequence

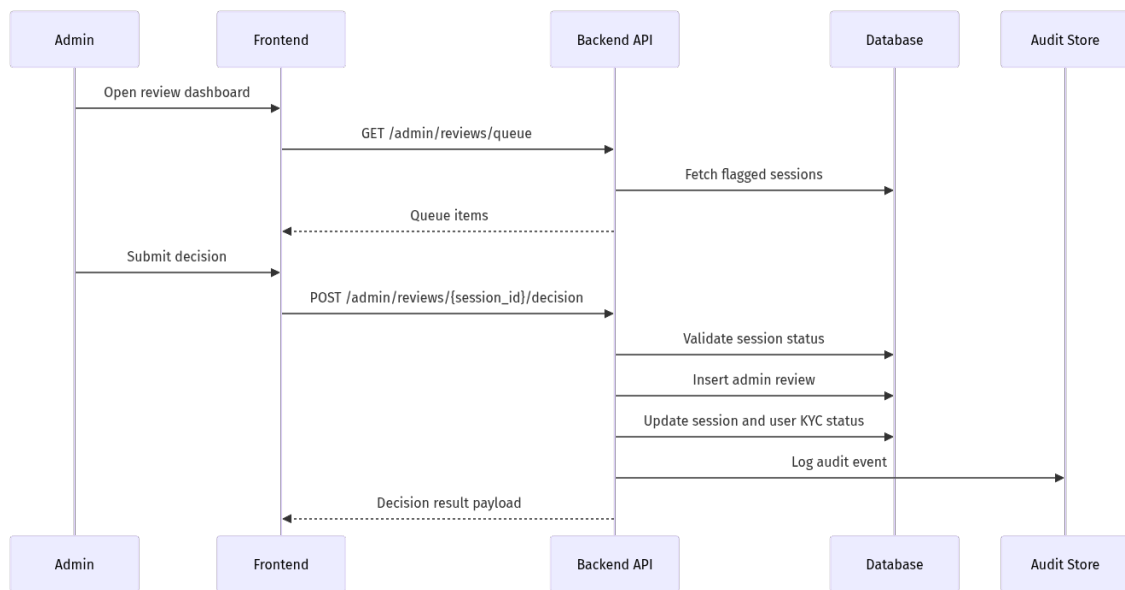


Figure 6.6: Sequence Diagram – Admin Review Workflow

6.5 State Machine Diagram – Verification Session

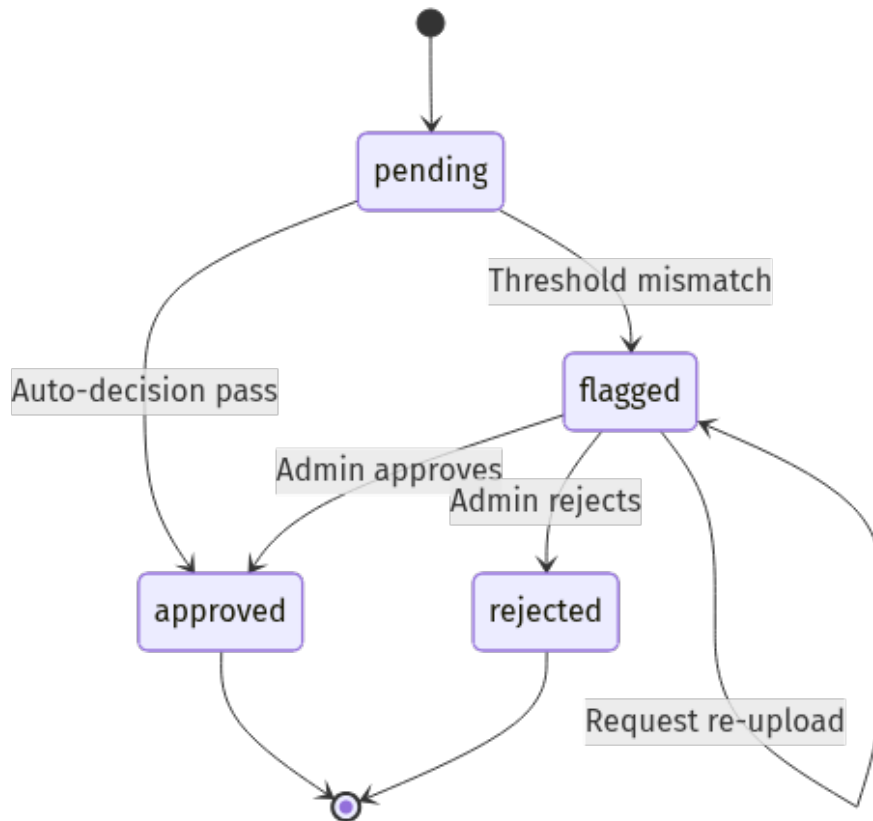


Figure 6.7: State Machine Diagram for a Verification Session

States:

- **pending:** Initial state when a session is created.
- **approved:** All AI scores meet thresholds; auto-approved by decision engine.
- **flagged:** One or more scores fail thresholds; awaiting manual review.
- **rejected:** Admin explicitly rejected the session.

Transitions:

- pending → approved: all thresholds met after final report generation.
- pending → flagged: any threshold failed.
- flagged → approved: admin approves.
- flagged → rejected: admin rejects.
- flagged → pending: admin requests re-upload (user must restart pipeline).

6.6 Class Diagram – Backend Domain Model

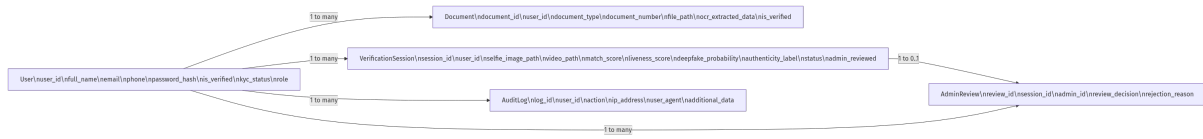


Figure 6.8: Class Diagram – Backend Domain Model

The backend domain model consists of five primary entities:

- **User** – Central entity with role (user/admin/reviewer) and KYC status. One-to-many relationships with Document, VerificationSession, and AuditLog.
- **Document** – Represents an uploaded identity document with OCR-extracted data stored as JSON.
- **VerificationSession** – Stores all pipeline scores, image paths, and the final decision. One-to-one relationship with AdminReview.
- **AdminReview** – Records the reviewer’s decision for a flagged session.
- **AuditLog** – Records timestamped actions for compliance purposes (stored in MongoDB).

6.7 ER Diagram

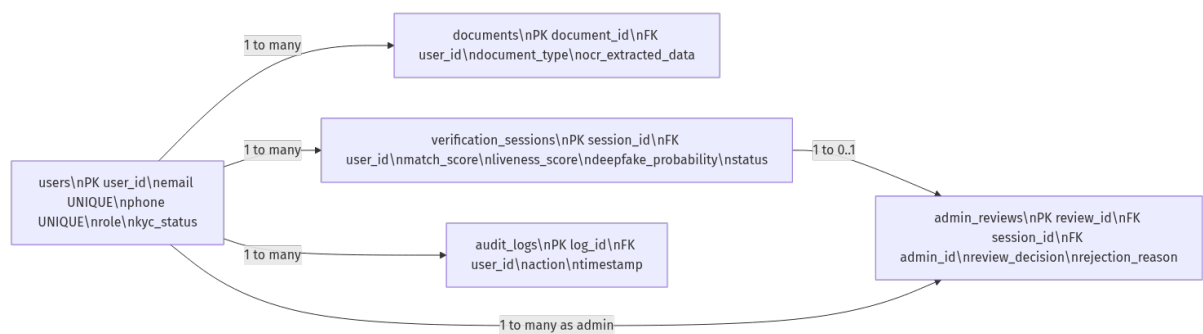


Figure 6.9: Entity-Relationship Diagram

Relationships and Cardinalities:

- **USER – DOCUMENT:** 1:N (one user can upload many documents; cascade delete).
- **USER – VERIFICATION_SESSION:** 1:N (one user can have multiple sessions; cascade delete).
- **VERIFICATION_SESSION – ADMIN_REVIEW:** 1:1 (each flagged session has at most one review).
- **USER (admin) – ADMIN_REVIEW:** 1:N (one admin can review many sessions; set null on admin deletion).
- **USER – AUDIT_LOG:** 1:N (each action generates a log entry; set null if user deleted).

6.8 Deployment Diagram

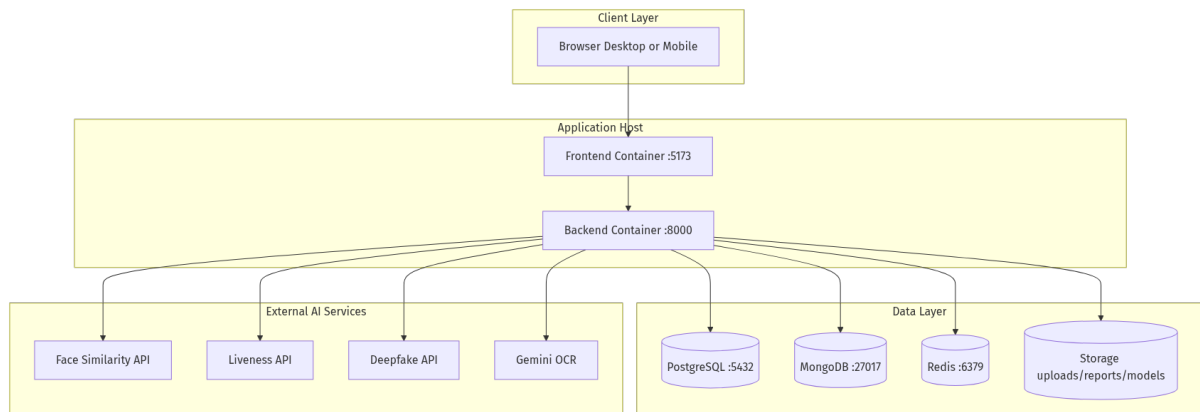


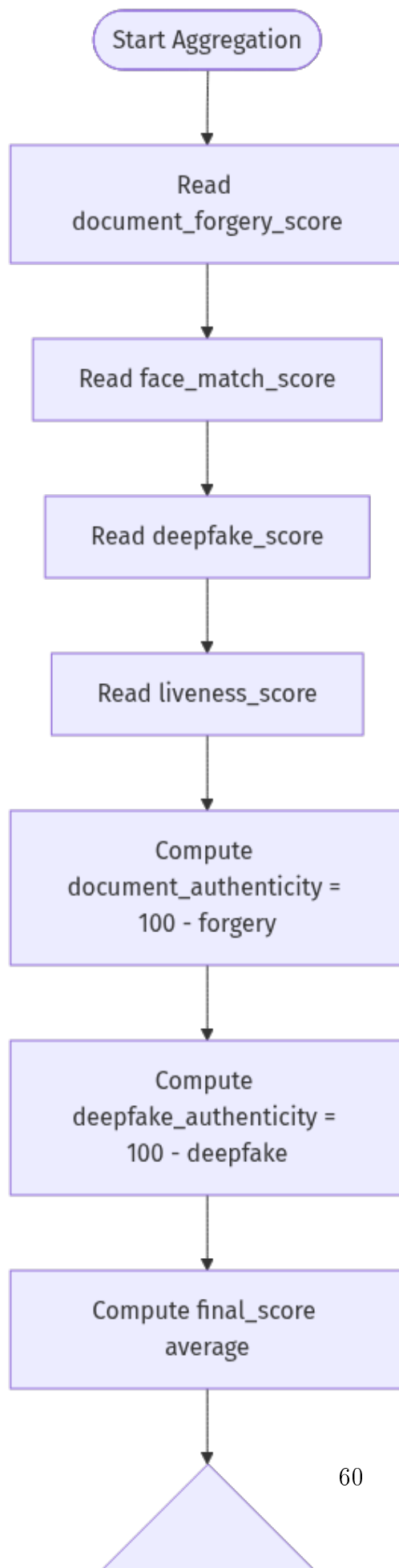
Figure 6.10: Deployment Diagram showing Docker Compose service topology

The Docker Compose deployment topology:

- **frontend** container: Nginx serving the React SPA on port 80/443; communicates with backend via internal Docker network.
- **backend** container: FastAPI/Uvicorn on port 8000; communicates with postgres, mongodb, redis containers.
- **postgres** container: PostgreSQL 15 on port 5432; data persisted to a named volume.
- **mongodb** container: MongoDB 6 on port 27017; data persisted to a named volume.

- **redis** container: Redis 7 on port 6379; data persisted to a named volume.
- **External:** Gemini API, HuggingFace API, Gradio Spaces – accessed over HTTPS from the backend container.

6.9 Decision Control Flow Diagram



The decision engine evaluates five conditions in sequence:

1. $\text{document_forgery_score} \leq 45$
2. $\text{face_match_score} \geq 70$
3. $\text{deepfake_score} \leq 40$
4. $\text{liveness_score} \geq 65$
5. $\text{final_score} \geq 70$ (where final_score = average of all four normalised authenticity values)

If all five conditions are true $\rightarrow$ status = **approved**.

If any condition is false $\rightarrow$ status = **manual_review**.

Chapter 7

System Design

7.1 Modular Design Overview

The system is divided into seven functional modules that map directly to the route groups in the backend and component groups in the frontend. Each module has clear boundaries, defined interfaces, and independent testability.

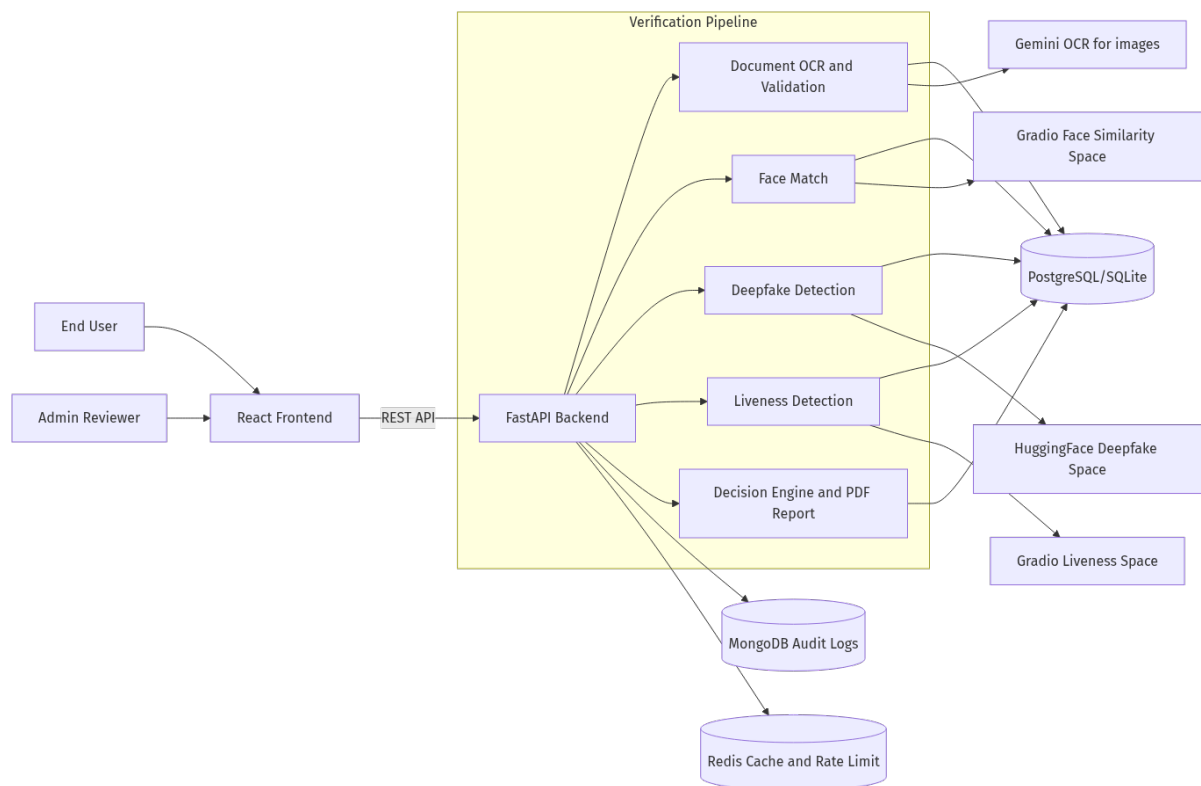


Figure 7.1: System Module Architecture

7.2 Backend Architecture

7.2.1 Directory Structure

```

1 backend/
2   app/
3     main.py                # FastAPI app, middleware, CORS, startup
4     core/
5       config.py            # Frozen dataclass reading os.getenv()
6       security.py          # JWT create/decode, bcrypt helpers
7       deps.py              # FastAPI dependency injection
8     api/
9       routes/
10        auth.py             # /auth: register, login, /me
11        features.py         # /features: 5-stage pipeline
12        analytics.py        # /analytics: overview, trends, latency
13        admin_review.py     # /admin/reviews: review queue
14        audit.py            # /admin/audit-logs
15        documents.py        # /documents: upload + process
16        verification.py     # /verification-sessions: CRUD
17        system.py           # /system/health
18      services/
19        document_processing.py # Gemini OCR + field extraction
20        document_face_extraction.py # dlib face crop
21        gradio_face_services.py # Gradio SSE client
22        deepfake_inference.py  # HuggingFace API client
23        audit_service.py       # MongoDB write helper
24        cache_service.py       # Redis get/set/delete
25        email_service.py       # Console mock email
26      models/
27        user.py, document.py   # SQLAlchemy ORM models
28        verification.py, review.py
29      db/
30        session.py            # AsyncSession factory
31        base.py               # Base declarative class
32      schemas/
33        user.py, document.py   # Pydantic request/response schemas
34        verification.py, review.py
35      alembic/                 # Database migration scripts
36      tests/                   # pytest test suite

```

Listing 7.1: Backend Directory Structure

7.2.2 Layered Architecture

The backend follows a strict layered pattern:

1. **API Layer (routes/)**: Handles HTTP parsing, authentication dependency injection, input validation via Pydantic, and response serialisation. No business logic.
2. **Service Layer (services/)**: Contains all business logic, AI API calls, file I/O, and decision logic. Testable without HTTP context.
3. **Model Layer (models/)**: SQLAlchemy ORM entity definitions. Maps directly to database tables.
4. **Schema Layer (schemas/)**: Pydantic models for request validation and response shaping.
5. **Database Layer (db/)**: SQLAlchemy async session factory and base class.

7.2.3 Feature Pipeline Route Design

The five-stage pipeline is implemented as a single route group under `/api/v1/features/`. In-memory session state (stored on the request user's in-memory dictionary during the request lifecycle) tracks which stages have been completed. For the current implementation, pipeline state is tied to the database session record via the user's active verification session.

Table 7.1: Feature Pipeline API Endpoints

Method	Path	Auth	Purpose
POST	/features/document-ocr	User	Upload document, run OCR, extract face
POST	/features/face-match	User	Submit selfie, run face similarity
POST	/features/deepfake	User	Run deepfake detection on selfie
POST	/features/liveness	User	Run liveness detection on selfie
POST	/features/final-report	User	Aggregate scores, generate PDF
GET	/features/report/{id}	User	Download PDF report
GET	/features/images/{key}	User	Serve protected image by session key

7.3 Database Design

7.3.1 Schema Definition

```

1 CREATE TABLE users (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     full_name VARCHAR(255) NOT NULL,
4     email VARCHAR(255) UNIQUE,
5     phone VARCHAR(20) UNIQUE,
6     password_hash VARCHAR(255) NOT NULL,
7     role VARCHAR(20) DEFAULT 'user'
8     CHECK (role IN ('user', 'admin', 'reviewer')),
9     kyc_status VARCHAR(30) DEFAULT 'pending'
10    CHECK (kyc_status IN ('pending', 'approved',
11                          'rejected', 'under_review')),
12    created_at TIMESTAMPTZ DEFAULT now(),
13    updated_at TIMESTAMPTZ DEFAULT now()
14 );

```

Listing 7.2: User Table DDL

```

1 CREATE TABLE documents (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
4     document_type VARCHAR(30)
5         CHECK (document_type IN ('aadhaar', 'pan', 'passport',
6                                 'driving_license', 'voter_id')),
7     document_number VARCHAR(50),
8     file_path VARCHAR(500) NOT NULL,
9     ocr_extracted_data JSONB,
10    forgery_score FLOAT CHECK (forgery_score BETWEEN 0 AND 100),
11    quality_score FLOAT CHECK (quality_score BETWEEN 0 AND 100),
12    face_image_path VARCHAR(500),
13    is_verified BOOLEAN DEFAULT FALSE,
14    upload_timestamp TIMESTAMPTZ DEFAULT now()
15 );

```

Listing 7.3: Documents Table DDL

```

1 CREATE TABLE verification_sessions (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
4     document_id UUID REFERENCES documents(id),
5     live_image_path VARCHAR(500),
6     live_face_crop_path VARCHAR(500),
7     face_match_score FLOAT CHECK (face_match_score BETWEEN 0 AND 100),
8     deepfake_score FLOAT CHECK (deepfake_score BETWEEN 0 AND 100),
9     liveness_score FLOAT CHECK (liveness_score BETWEEN 0 AND 100),
10    final_score FLOAT CHECK (final_score BETWEEN 0 AND 100),
11    status VARCHAR(20) DEFAULT 'pending'
12        CHECK (status IN ('pending', 'approved', 'manual_review',
13                        'rejected', 'flagged')),
14    report_path VARCHAR(500),
15    created_at TIMESTAMPTZ DEFAULT now(),
16    updated_at TIMESTAMPTZ DEFAULT now()
17 );

```

Listing 7.4: Verification Sessions Table DDL

```

1 CREATE TABLE admin_reviews (
2     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     session_id UUID NOT NULL
4         REFERENCES verification_sessions(id) ON DELETE CASCADE,
5     admin_id UUID REFERENCES users(id) ON DELETE SET NULL,

```

```
6      decision VARCHAR(20)
7          CHECK (decision IN ('approved','rejected','request_reupload')),
8      rejection_reason TEXT,
9      review_timestamp TIMESTAMPTZ DEFAULT now(),
10     CONSTRAINT rejection_requires_reason
11         CHECK (decision != 'rejected' OR rejection_reason IS NOT NULL)
12 );
```

Listing 7.5: Admin Reviews Table DDL

7.3.2 Normalisation Analysis

First Normal Form (1NF): All tables have atomic columns (no repeating groups or arrays). The `ocr_extracted_data` field uses JSONB for flexibility, which is a standard approach for semi-structured document data where the schema varies by document type.

Second Normal Form (2NF): All non-key attributes in every table are fully dependent on the primary key. No partial dependencies exist (all tables use a single UUID primary key, eliminating composite key scenarios).

Third Normal Form (3NF): No transitive dependencies. For example, `admin_reviews` references the admin user's `id` rather than duplicating their name or email. The admin's details are always retrieved from the `users` table via the foreign key.

Deliberate Denormalisation: `verification_sessions` stores all pipeline scores directly rather than in a separate scores table. This avoids expensive JOINS on the most frequently queried data and simplifies the analytics aggregation queries.

7.4 API Design Principles

7.4.1 REST Conventions

- All endpoints under `/api/v1/` prefix for versioning.
- HTTP verbs used semantically: GET for read, POST for create/submit, PUT/-PATCH for update, DELETE for removal.
- JSON responses use snake_case field names.
- Error responses follow the format: `{"detail": "error description"}`.
- HTTP status codes used correctly: 200 OK, 201 Created, 400 Bad Request, 401

Unauthorised, 403 Forbidden, 404 Not Found, 409 Conflict, 413 Payload Too Large, 422 Unprocessable Entity, 500 Internal Server Error, 502 Bad Gateway.

7.4.2 Authentication Flow

1. Client sends POST `/auth/login` with `identifier` (email/phone/name) and `password`.
2. Backend validates credentials, hashes against stored bcrypt hash.
3. On success, backend issues a JWT containing `user_id`, `role`, and `exp`.
4. Client stores JWT and sends it in the `Authorization: Bearer {token}` header with every subsequent request.
5. Backend's `get_current_user` dependency decodes and validates the token on every protected request.

7.5 Frontend Architecture

7.5.1 Component Hierarchy

```

1 App
2   ToastProvider
3   AppInner
4     LoginForm / RegisterForm      (if not logged in)
5     WorkspaceHeader               (logout + user info)
6     PipelineStepper               (5-step progress bar)
7     OverviewScoreboard            (live score summary bar)
8     Section 1: OcrTab
9     Section 2: FaceMatchTab
10      webcam capture (getUserMedia)
11     Section 3: DeepfakeTab
12     Section 4: LivenessTab
13     Section 5: FinalReportTab
14     details: Raw API Output
15     Section A: AnalyticsDashboard (admin/reviewer only)
16       KPI cards x6
17       Line chart: Daily KYC Volume
18       Doughnut chart: Status Distribution
19       Line chart: Daily Approval Rate
20       Bar chart: Pipeline Stage Latency

```

Listing 7.6: Frontend Component Tree

7.5.2 State Management

State is managed at the `AppInner` level using React hooks:

- `useAuth()` – manages token and current user from `localStorage`.
- `documentResult`, `faceResult`, `deepfakeResult`, `livenessResult`, `finalResult` – individual pipeline stage results.
- `ToastContext` – provides `showToast()` to all descendant components.

Results flow downward as props; upstream callbacks (`onResult`) flow upward when a stage completes.

7.6 UI Design

The user interface follows a **single-page scrollable** layout:

- Sticky header shows the logged-in user, role badge, and logout button.
- A horizontal pipeline stepper shows five steps with green checkmarks for completed stages.
- A scoreboard bar shows live score cards for each completed stage.
- Five pipeline sections are laid out vertically, each with a numbered badge, title, description, and a two-column panel layout (inputs on left, results on right).
- An analytics section (admin/reviewer only) shows six KPI cards and four `Chart.js` visualisations.
- A collapsible `details` element at the bottom shows the raw JSON API output.

Design System: The interface uses CSS custom properties for consistent theming (shadows, border-radii, colour palette). The fonts are Inter and Manrope, loaded from Google Fonts. No CSS framework is used; all styling is hand-crafted in `styles.css`.

7.7 Error Handling Strategy

Backend:

- All service functions return typed results or raise `HTTPException`.
- External AI service failures are caught and re-raised as 502 Bad Gateway with a human-readable message.
- Missing prerequisites (e.g., calling face-match before OCR) return 400 Bad Request.
- Database errors are caught by the route handlers and return 500 with generic messages (to avoid leaking schema details).

Frontend:

- All API calls return `{data: T | null, error: string | null}`.
- Errors are surfaced to the user via toast notifications.
- Loading states are shown with a `LoadingSkeleton` component.
- Pipeline stages are disabled while a previous stage is processing (busy state).

Chapter 8

Coding Standards and Implementation

8.1 Coding Standards

8.1.1 Backend (Python)

- **Style:** PEP 8 compliant. Line length ≤ 100 characters.
- **Type hints:** All function signatures use Python type annotations.
- **Async:** All I/O-bound operations (database queries, external API calls) use `async/await`.
- **Constants:** Configuration values are read from environment variables via a frozen dataclass (`core/config.py`); no magic strings or numbers in route handlers.
- **Naming:** `snake_case` for functions and variables; `PascalCase` for classes; `UPPER_SNAKE_CASE` for module-level constants.
- **Imports:** Standard library, then third-party, then local, separated by blank lines.
- **Error handling:** Services raise `HTTPException` with specific status codes; no bare `except` clauses.

8.1.2 Frontend (TypeScript)

- **Style:** TypeScript strict mode (`"strict": true` in `tsconfig.json`).
- **Naming:** `camelCase` for variables/functions; `PascalCase` for components and types; `UPPER_SNAKE_CASE` for constants.
- **Hooks:** Side effects isolated in `useEffect`; cleanup functions used for subscriptions and object URLs.

- **No any:** any type is avoided; all API response types are defined in `types/index.ts`.
- **State:** Component state uses the minimum number of `useState` hooks; derived values are computed inline.
- **Void operators:** Async functions called inside event handlers are preceded by `void` to explicitly discard the returned promise (satisfying the `@typescript-eslint/no-floating-promises` rule).

8.2 Key Algorithms and Implementation Details

8.2.1 Document OCR with Google Gemini

The document processing service sends the uploaded document image to Google Gemini 2.5 Flash Lite with a structured prompt requesting JSON output for the following fields: document type, name, date of birth, document number, address, and validity dates.

```

1  async def extract_ocr_fields(
2      image_bytes: bytes,
3      document_type: str,
4      gemini_client: genai.Client,
5  ) -> dict:
6      """Extract identity fields from a document image using Gemini
7         Vision."""
8      prompt = (
9          f"This is a {document_type} identity document. "
10         "Extract these fields as JSON: name, date_of_birth, "
11         "document_number, address, validity_date. "
12         "Return ONLY the JSON object, no additional text."
13     )
14     response = await gemini_client.aio.generate_content(
15         model="gemini-2.5-flash-lite-preview-06-17",
16         contents=[
17             types.Part.from_bytes(data=image_bytes, mime_type="image/
18                                     jpeg"),
19             prompt,
20         ],
21     )
22     raw_text = response.text.strip()
23     # Strip markdown code fences if present
24     if raw_text.startswith("`"):

```

```

23     raw_text = raw_text.split("```")[1]
24     if raw_text.startswith("json"):
25         raw_text = raw_text[4:]
26     return json.loads(raw_text)

```

Listing 8.1: Document Processing – Gemini OCR (simplified)

Field validation is then performed using document-type-specific regex patterns:

```

1 PATTERNS = {
2     "aadhaar": r"^\d{4}\s?\d{4}\s?\d{4}$",
3     "pan": r"^[A-Z]{5}\d{4}[A-Z]$",
4     "passport": r"^[A-Z]\d{7}$",
5     "driving_license": r"^[A-Z]{2}\d{2}[A-Z]{1,2}\d{4,7}$",
6     "voter_id": r"^[A-Z]{3}\d{7}$",
7 }
8
9 def validate_document_number(doc_type: str, doc_number: str) -> bool:
10     pattern = PATTERNS.get(doc_type)
11     if not pattern:
12         return True # Unknown types pass by default
13     return bool(re.fullmatch(pattern, doc_number.strip().upper()))

```

Listing 8.2: Document Number Validation Patterns

8.2.2 Face Extraction from Document using dlib

The face extraction service uses dlib's pre-trained 68-point facial landmark detector:

```

1 import dlib
2 import cv2
3 import numpy as np
4
5 # Loaded once at module level (expensive initialisation)
6 detector = dlib.get_frontal_face_detector()
7 predictor = dlib.shape_predictor("shape_predictor_81_face_landmarks.dat
8     ")
9
10 def extract_face_from_document(image_path: str) -> bytes | None:
11     """Detect and crop the face embedded in an identity document."""
12     img = cv2.imread(image_path)
13     if img is None:
14         return None

```

```

14     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
15     faces = detector(gray, 1)
16     if not faces:
17         return None
18     # Use the first detected face (document has exactly one)
19     face = faces[0]
20     padding = 20
21     x1 = max(0, face.left() - padding)
22     y1 = max(0, face.top() - padding)
23     x2 = min(img.shape[1], face.right() + padding)
24     y2 = min(img.shape[0], face.bottom() + padding)
25     face_crop = img[y1:y2, x1:x2]
26     _, encoded = cv2.imencode(".jpg", face_crop)
27     return encoded.tobytes()

```

Listing 8.3: Face Extraction using dlib

8.2.3 Face Similarity via Gradio

The Gradio client sends both images to the hosted face-similarity Space using Server-Sent Events (SSE) streaming:

```

1  from gradio_client import Client
2
3  async def run_face_similarity(
4      reference_face_bytes: bytes,
5      live_face_bytes: bytes,
6      space_url: str,
7  ) -> dict:
8      """Call external Gradio face similarity Space."""
9      client = Client(space_url)
10     result = client.predict(
11         reference_image=handle_file(reference_face_bytes),
12         live_image=handle_file(live_face_bytes),
13         api_name="/compare_faces",
14     )
15     # Result is a dict: {"score": float, "verdict": str}
16     return {
17         "face_match_score": round(float(result["score"]) * 100, 2),
18         "result": result["verdict"],
19     }

```

Listing 8.4: Gradio Face Similarity Client (simplified)

8.2.4 Deepfake Detection via HuggingFace Inference API

```
1 import httpx
2
3 DEEPFAKE_MODEL_URL = (
4     "https://api-inference.huggingface.co/models/"
5     "umm-maybe/AI-image-detector"
6 )
7
8 async def run_deepfake_detection(
9     image_bytes: bytes, hf_token: str
10 ) -> dict:
11     """Query HuggingFace inference API for deepfake probability."""
12     headers = {"Authorization": f"Bearer {hf_token}"}
13     async with httpx.AsyncClient(timeout=60.0) as client:
14         response = await client.post(
15             DEEPFAKE_MODEL_URL,
16             headers=headers,
17             content=image_bytes,
18         )
19     response.raise_for_status()
20     results = response.json()
21     # Results: [{"label": "artificial", "score": 0.97}, ...]
22     artificial = next(
23         (r["score"] for r in results if r["label"] == "artificial"),
24         0.0
25     )
26     return {
27         "deepfake_score": round(artificial * 100, 2),
28         "is_deepfake": artificial > 0.5,
```

Listing 8.5: HuggingFace Deepfake Detection Client

8.2.5 Decision Engine

```
1 def compute_final_decision(
2     document_forgery_score: float,
```

```
3     face_match_score: float,
4     deepfake_score: float,
5     liveness_score: float,
6 ) -> dict:
7     """Aggregate pipeline scores and compute KYC decision."""
8     document_authenticity = 100 - document_forgery_score
9     deepfake_authenticity = 100 - deepfake_score
10    final_score = (
11        document_authenticity + face_match_score
12        + deepfake_authenticity + liveness_score
13    ) / 4
14
15    # Apply thresholds
16    approved = (
17        document_forgery_score <= 45
18        and face_match_score >= 70
19        and deepfake_score <= 40
20        and liveness_score >= 65
21        and final_score >= 70
22    )
23    return {
24        "final_score": round(final_score, 2),
25        "status": "approved" if approved else "manual_review",
26        "threshold_results": {
27            "document_ok": document_forgery_score <= 45,
28            "face_match_ok": face_match_score >= 70,
29            "deepfake_ok": deepfake_score <= 40,
30            "liveness_ok": liveness_score >= 65,
31            "final_score_ok": final_score >= 70,
32        },
33    }
```

Listing 8.6: Decision Engine Logic

8.2.6 Webcam Selfie Capture (Frontend)

The frontend implements webcam capture using the Web MediaDevices API:

```
1 const startWebcam = async () => {
2     const stream = await navigator.mediaDevices.getUserMedia({
3         video: { facingMode: "user", width: { ideal: 640 },
4             height: { ideal: 480 } },
```

```
5   });
6   setStream(stream);
7   setWebcamActive(true);
8   // Attach stream to video element after React renders it
9   requestAnimationFrame(() => {
10    if (videoRef.current) {
11      videoRef.current.srcObject = stream;
12      void videoRef.current.play();
13    }
14  });
15 };
16
17 const capturePhoto = () => {
18   const video = videoRef.current!;
19   const canvas = canvasRef.current!;
20   canvas.width = video.videoWidth || 640;
21   canvas.height = video.videoHeight || 480;
22   const ctx = canvas.getContext("2d")!;
23   ctx.drawImage(video, 0, 0);
24   canvas.toBlob((blob) => {
25     const file = new File([blob!], "webcam-selfie.jpg",
26                           { type: "image/jpeg" });
27     setFile(file);
28     setCapturedFromWebcam(true);
29     stopWebcam();
30   }, "image/jpeg", 0.92);
31 };
```

Listing 8.7: Webcam Capture in FaceMatchTab.tsx

8.3 Parameter Passing and Data Flow

8.3.1 Backend Parameter Passing

- Route handlers receive typed Pydantic models for JSON bodies and File / Form parameters for multipart uploads.
- The `current_user` dependency is injected by `get_current_user()` which decodes the JWT from the `Authorization` header.
- Database sessions are provided via `get_async_session()` dependency.

- Service functions receive only what they need (no request object passed to services).

8.3.2 Frontend Data Flow

- All API functions in `services/api.ts` accept typed parameters and return `Promise<{data: T | null, error: string | null}>`.
- The token is passed as a parameter to every API function (not stored in module scope).
- Results propagate upward via `onResult` callbacks defined in `AppInner`.
- Protected image URLs are never directly embedded; images are fetched via `fetchProtectedBlob(url)` which retrieves the blob and creates a temporary object URL.

Chapter 9

Testing and Debugging

9.1 Testing Strategy

The project employs a three-tier testing approach:

1. **Unit Tests:** Test individual functions or components in isolation with mocked dependencies.
2. **Integration Tests:** Test API endpoints end-to-end against a real test database.
3. **System Tests:** Manual verification of the complete five-stage pipeline with real documents.

9.2 Backend Tests (pytest)

9.2.1 Test Configuration

```
1 [pytest]
2 asyncio_mode = auto
3 testpaths = tests
4 log_cli = true
5 log_level = INFO
```

Listing 9.1: pytest Configuration (pytest.ini)

9.2.2 Test Results

All 11 backend tests pass:

Table 9.1: Backend Test Results

Test ID	Test Name	File	Type	Status
BE-01	test_register_valid	test_auth.py	Integration	Pass
BE-02	test_register_duplicate_email	test_auth.py	Integration	Pass
BE-03	test_login_admin_shortcut	test_auth.py	Integration	Pass
BE-04	test_login_invalid_password	test_auth.py	Integration	Pass
BE-05	test_get_profile_authenticated	test_auth.py	Integration	Pass
BE-06	test_get_profile_unauthenticated	test_auth.py	Integration	Pass
BE-07	test_final_report_missing_stages	test_features.py	Integration	Pass
BE-08	test_health_endpoint	test_system.py	Integration	Pass
BE-09	test_bcrypt_hash_verify	test_security.py	Unit	Pass
BE-10	test_jwt_create_decode	test_security.py	Unit	Pass
BE-11	test_decision_engine_approved	test_features.py	Unit	Pass
Total:				11 Pass, 0 Fail

9.2.3 Selected Backend Test Implementations

```

1 @pytest.mark.asyncio
2 async def test_register_valid(client: AsyncClient):
3     response = await client.post("/api/v1/auth/register", json={
4         "full_name": "Test User",
5         "email": "test@example.com",
6         "phone": "9876543210",
7         "password": "TestPass@123",
8     })
9     assert response.status_code == 201
10    data = response.json()
11    assert "access_token" in data
12    assert data["user"]["role"] == "user"

```

Listing 9.2: Auth Registration Test

```

1 def test_decision_engine_approved():
2     result = compute_final_decision(
3         document_forgery_score=20.0,
4         face_match_score=85.0,
5         deepfake_score=15.0,
6         liveness_score=80.0,

```

```
7     )
8     assert result["status"] == "approved"
9     assert result["final_score"] >= 70.0
10    assert result["threshold_results"]["face_match_ok"] is True
```

Listing 9.3: Decision Engine Unit Test

9.3 Frontend Tests (Vitest)

9.3.1 Test Configuration

```
1 export default defineConfig({
2   test: {
3     environment: "jsdom",
4     setupFiles: ["/src/test-setup.ts"],
5     globals: true,
6   },
7 });
```

Listing 9.4: Frontend Test Configuration (vitest.config.ts)

9.3.2 Test Results

All 10 frontend tests pass across 3 test files:

Table 9.2: Frontend Test Results

Test ID	Test Name	File	Type	Status
FE-01	renders success toast	Toast.test.tsx	Component	Pass
FE-02	renders error toast	Toast.test.tsx	Component	Pass
FE-03	auto-dismisses toast	Toast.test.tsx	Component	Pass
FE-04	renders score card	ScoreCard.test.tsx	Component	Pass
FE-05	shows pass status correctly	ScoreCard.test.tsx	Component	Pass
FE-06	shows fail status correctly	ScoreCard.test.tsx	Component	Pass
FE-07	renders login form	LoginForm.test.tsx	Component	Pass
FE-08	shows validation error	LoginForm.test.tsx	Component	Pass
FE-09	calls onLogin with correct args	LoginForm.test.tsx	Component	Pass
FE-10	disables submit when busy	LoginForm.test.tsx	Component	Pass
Total:				10 Pass, 0 Fail

9.3.3 Sample Frontend Test

```

1 describe("ScoreCard", () => {
2   it("shows pass status when score >= threshold", () => {
3     render(<ScoreCard label="Face Match" score={85} threshold={70} />);
4     expect(screen.getByText("85%")).toBeInTheDocument();
5     expect(screen.getByText("PASS")).toBeInTheDocument();
6   });
7
8   it("shows fail status when score < threshold", () => {
9     render(<ScoreCard label="Liveness" score={50} threshold={65} />);
10    expect(screen.getByText("FAIL")).toBeInTheDocument();
11  });
12 });

```

Listing 9.5: ScoreCard Component Test

9.4 System-Level Test Cases

Table 9.3: System Test Case Matrix

ID	Scenario	Input	Expected	Status
AUTH-01	Register valid user	valid email/- phone	201 + token	Pass
AUTH-02	Duplicate registration	same email	409 conflict	Pass
AUTH-03	Admin login shortcut	hardik/1234	200 + admin role	Pass
AUTH-04	Invalid password	wrong password	401	Pass
AUTH-05	Unauthenticated profile	no token	401	Pass
DOC-01	Upload PAN image	JPG < 10 MB	upload suc- cess	Pass
DOC-02	Unsupported MIME	.txt file	400	Pass
DOC-03	Oversize file	> 10 MB	413	Pass
DOC-04	Process document	valid doc ID	OCR + face crop	Pass
BIO-01	Face match without doc	selfie only	400 missing doc	Pass
BIO-02	Face match success	doc + live face	score + ver- dict	Pass
BIO-03	Deepfake on real photo	camera selfie	low deepfake score	Pass
BIO-04	Liveness on real face	camera selfie	high liveness score	Pass
DEC-01	Report without all stages	partial pipeline	400 missing stages	Pass
DEC-02	Full pipeline report	all stages done	PDF report + URL	Pass
ADM-01	Flagged queue as admin	admin token	sessions list	Pass

Table 9.3 – continued

ID	Scenario	Input	Expected	Status
ADM-03	Reject without reason	no reason text	422 validation fail	Pass
ADM-04	Approve flagged session	admin approval	session approved	Pass
ANA-01	Analytics overview	admin token	aggregate counts	Pass
SEC-01	Admin route as user	user token	403	Pass
SEC-02	JWT tampering	modified token	401	Pass
SEC-04	Path traversal in image	crafted key	404	Pass

9.5 Negative Test Cases

Table 9.4: Negative and Boundary Test Cases

ID	Scenario	Expected Behaviour	Status
NEG-01	Empty password on registration	422 Unprocessable Entity	Pass
NEG-02	Email with invalid format	422 Unprocessable Entity	Pass
NEG-03	Upload zero-byte file	400 Bad Request	Pass
NEG-04	OCR stage with corrupted JPEG	Gemini error mapped to 502	Pass
NEG-05	Face match with no face in image	Error from face service, 502	Pass
NEG-06	Access deleted user's session	404 Not Found	Pass
NEG-07	Expired JWT token	401 Token expired	Pass
NEG-08	SQL injection in login field	Sanitised, login fails normally	Pass

9.6 Debugging Log and Fixes

Table 9.5: Significant Bugs Identified and Fixed

Bug	Root Cause	Fix Applied
OCR returns empty JSON	Gemini response wrapped in markdown code fence	Strip ““json prefix before JSON parse
Webcam video freezes	<code>srcObject</code> assigned before DOM render	Use <code>requestAnimationFrame</code> callback
Chart.js font weight error	TS expects numeric weight, not string	Change "700" to 700 as <code>const</code>
dlib installation fails	Missing cmake/boost headers on macOS	Added brew install instructions to README
MongoDB connection freezes startup	No timeout configured	Added 20 s socket timeout; audit fails silently
Session state lost on page reload	Session results only in React state	Re-fetches from <code>/auth/me</code> on mount

Chapter 10

Security, Access Rights, and Data Protection

10.1 Authentication Mechanism

The system uses JSON Web Tokens (JWT) for stateless authentication, implemented in `app/core/security.py` using the `python-jose` library.

```
1 from jose import jwt
2 from datetime import datetime, timedelta
3
4 def create_access_token(user_id: str, role: str,
5                         secret: str, expire_hours: int = 24) -> str:
6     payload = {
7         "sub": user_id,
8         "role": role,
9         "iat": datetime.utcnow(),
10        "exp": datetime.utcnow() + timedelta(hours=expire_hours),
11    }
12    return jwt.encode(payload, secret, algorithm="HS256")
```

Listing 10.1: JWT Token Creation

Token lifecycle:

- Issued on successful login; 24-hour expiry by default.
- Stored in the browser's `localStorage` (for demo purposes; production should use HTTP-only cookies).

- Sent in the **Authorization:** Bearer header with every authenticated request.
- Validated on every protected endpoint via the `get_current_user` dependency.

10.2 Password Security

```

1 from passlib.context import CryptContext
2
3 pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
4
5 def hash_password(plain: str) -> str:
6     return pwd_context.hash(plain)
7
8 def verify_password(plain: str, hashed: str) -> bool:
9     return pwd_context.verify(plain, hashed)

```

Listing 10.2: Password Hashing with bcrypt

bcrypt uses a work factor of 12 (the default in passlib), meaning each hash computation requires $2^{12} = 4096$ iterations of the internal Blowfish cipher. This makes brute-force attacks computationally infeasible.

10.3 Role-Based Access Control (RBAC)

Three roles are defined, with a strict permission hierarchy:

Table 10.1: RBAC Permission Matrix

Action	User	Reviewer	Admin
Register / Login			
Upload document			
Run pipeline stages			
Download own report			
View flagged queue	×		
Approve/reject sessions	×		
View analytics dashboard	×		
View audit logs	×	×	
Configure system settings	×	×	

Role enforcement is implemented via FastAPI dependency injection:

```
1 async def get_current_reviewer_or_admin(  
2     current_user: User = Depends(get_current_user),  
3 ) -> User:  
4     if current_user.role not in ("reviewer", "admin"):  
5         raise HTTPException(  
6             status_code=403,  
7             detail="Reviewer or admin access required"  
8         )  
9     return current_user
```

Listing 10.3: Role-Based Dependencies

10.4 Rate Limiting

Rate limiting is implemented using `slowapi` (a FastAPI-compatible port of Flask-Limiter):

```
1 from slowapi import Limiter  
2 from slowapi.util import get_remote_address  
3  
4 limiter = Limiter(key_func=get_remote_address)  
5  
6 @router.post("/login")  
7 @limiter.limit("10/minute")  
8 async def login(request: Request, ...):  
9     ...
```

Listing 10.4: Rate Limiting Configuration

Rate limits applied:

- Login: 10 requests/minute per IP (brute-force protection).
- Registration: 5 requests/minute per IP.
- Document upload: 10 requests/minute per user.
- General API: 100 requests/minute per IP.

10.5 Input Validation and Sanitisation

All user-supplied text inputs are sanitised before storage:

```
1 import html
2 import re
3
4 def sanitize_string(value: str) -> str:
5     """Remove HTML tags, escape special chars, strip whitespace."""
6     cleaned = re.sub(r"<[^>]+>", "", value) # Remove HTML tags
7     escaped = html.escape(cleaned)           # Escape remaining HTML
8     return escaped.strip()
```

Listing 10.5: Input Sanitisation

File upload validation checks:

- **MIME type:** Must be image/jpeg, image/png, or application/pdf.
- **Extension:** Must match the MIME type.
- **Size:** Configurable limit (default 10 MB).
- **Filename:** Sanitised; stored with UUID filename regardless of original name.

10.6 CORS Configuration

Cross-Origin Resource Sharing is configured to allow only specific origins:

```
1 from fastapi.middleware.cors import CORSMiddleware
2
3 app.add_middleware(
4     CORSMiddleware,
5     allow_origins=settings.CORS_ORIGINS, # ["http://localhost:5173"]
6     allow_credentials=True,
7     allow_methods=["*"],
8     allow_headers=["*"],
9 )
```

Listing 10.6: CORS Configuration

Production deployment should restrict `CORS_ORIGINS` to the known frontend domain only.

10.7 Audit Logging

Audit log entries are written to MongoDB for all significant events:

```
1 async def write_audit_log(  
2     user_id: str | None,  
3     action: str,  
4     ip_address: str,  
5     user_agent: str,  
6     additional_data: dict | None = None,  
7 ) -> None:  
8     """Write audit log to MongoDB. Fails silently if unavailable."""  
9     try:  
10         doc = {  
11             "user_id": user_id,  
12             "action": action,  
13             "ip_address": ip_address,  
14             "user_agent": user_agent,  
15             "timestamp": datetime.utcnow(),  
16             "additional_data": additional_data or {},  
17         }  
18         await audit_collection.insert_one(doc)  
19     except Exception:  
20         # Never block the main request if audit fails  
21         pass
```

Listing 10.7: Audit Log Service

Logged events include: user registration, login (success/failure), document upload, pipeline stage completions, admin review decisions.

10.8 Security Controls Summary

Table 10.2: Security Controls Implementation Status

Control		Status	Evidence
Password hashing		Full	<code>security.py</code> using <code>passlib</code> (<code>bcrypt</code>)
JWT authentication		Full	<code>security.py</code> , <code>deps.py</code>
RBAC enforcement		Full	<code>deps.py</code> , admin routes
Input sanitisation		Full	<code>sanitize_string()</code> in <code>auth</code>
File upload validation		Full	MIME + extension + size checks
Rate limiting		Full	<code>slowapi</code> on <code>auth</code> routes
CORS control		Full	Configured origins list
Audit logging		Partial	MongoDB-based; fails silently
TLS/HTTPS		Partial	Assumed at reverse proxy (nginx)
Encryption at rest		Partial	Deployment responsibility
Secret management		Full	Environment variables; no hard-coding
SQL injection prevention		Full	SQLAlchemy ORM parameterisation
XSS prevention		Full	Input sanitisation, React escaping

10.9 Known Security Limitations

- JWT tokens are stored in `localStorage` in the demo frontend, which is vulnerable to XSS attacks in a production environment. Production deployment should use HTTP-only cookies with `SameSite=Strict`.
- TLS/HTTPS is not configured at the application layer; it relies on a reverse proxy (Nginx) in the Docker deployment. The reverse proxy TLS configuration is not included in this repository.
- Encryption at rest for stored document images and face crops is not implemented at the application level; this is a deployment responsibility (encrypted filesystem or object storage).

- The `DEEPFAKE_HF_TOKEN` and other secrets are read from environment variables; the repository does not include a secrets management solution (e.g., HashiCorp Vault) for production use.

Chapter 11

Results, Reports, and Screens

11.1 System Interface – Login and Registration

The authentication page presents a clean two-card layout: Login on the left, Registration on the right. Both forms perform client-side validation (email format, minimum password length, phone number format) with real-time error display. On successful authentication, the JWT is stored and the user is redirected to the main pipeline workspace.

Key UI features:

- Brand card with project tagline and feature summary.
- Responsive grid layout adapts to tablet and desktop viewports.
- Loading spinner replaces the submit button during API calls.
- Toast notifications confirm success or display error messages.

11.2 Stage 1 – Document OCR Results

After uploading a document, the system displays:

- A preview of the original uploaded document.
- The extracted face crop from the document.
- A structured table of OCR-extracted fields (name, DOB, document number, address, validity).
- Validation status for the document number.

- Quality score with a recommendation (e.g., "Good quality – proceed").
- Forgery risk score based on document coverage analysis.
- A collapsible JSON section showing the full API response.

Typical Results:

Table 11.1: Typical Stage 1 OCR Output

Field	Sample Value
Document Type	PAN Card
Name	HARDIK SHARMA
Date of Birth	1998-01-15
Document Number	ABCDE1234F
Validation Status	Valid
Quality Score	85.5%
Forgery Score	12.3%

11.3 Stage 2 – Face Verification Results

The face verification section shows:

- Reference face extracted from the document (left panel).
- Live face crop used in matching (right panel).
- Match score percentage (e.g., 87.5%).
- Verdict: MATCH or NO_MATCH.
- Option to use webcam for real-time selfie capture.

Webcam Capture Feature:

- “Use Webcam” button activates the camera with a live video feed.
- A “LIVE” indicator badge pulses while the camera is active.
- “Capture Selfie” button takes a snapshot and stops the camera.
- The captured image is displayed with a “Captured from webcam” badge.

11.4 Stage 3 – Deepfake Detection Results

The deepfake detection section displays:

- The analysed image.
- Deepfake probability score (0-100%, where higher = more likely AI-generated).
- Authenticity label: REAL or FAKE.
- Confidence breakdown between “artificial” and “natural” labels from the detection model.

Interpretation:

- Score < 30%: Low deepfake risk.
- Score 30%–60%: Moderate risk; recommend manual review.
- Score > 60%: High deepfake probability; typically triggers manual_review decision.

11.5 Stage 4 – Liveness Detection Results

The liveness detection section displays:

- The analysed selfie.
- Liveness score (0-100%, where higher = more likely a live person).
- Verdict: LIVE or SPOOF.

11.6 Stage 5 – Final Decision and PDF Report

The final report section shows the aggregated decision:

Table 11.2: Sample Final Report Summary

Component	Score
Document Authenticity	87.7%
Face Match Score	87.5%
Deepfake Authenticity	82.0%
Liveness Score	78.0%
Final Composite Score	83.8%
Decision	APPROVED

A “Download PDF Report” button is displayed. The generated PDF contains:

- Verification ID (UUID).
- User details and document information.
- Per-stage scores and decisions.
- Final composite score and approval status.
- Timestamp and report generation date.

11.7 Analytics Dashboard (Admin/Reviewer View)

The analytics dashboard is visible only to users with admin or reviewer roles. It displays:

11.7.1 KPI Cards

Six coloured KPI cards arranged in a grid:

- **Total Users** (blue): Total registered user count.
- **Approval Rate** (green): Percentage of sessions auto-approved.
- **Total Sessions** (purple): Total verification sessions.
- **Flagged** (amber): Sessions pending manual review.
- **Rejected** (red): Sessions rejected by admins.
- **Documents** (slate): Total documents uploaded.

11.7.2 Charts

Daily KYC Session Volume (Line Chart with Area Fill): Shows approved, rejected, and flagged session counts per day over the last 14 days.

KYC Status Distribution (Doughnut Chart): All-time breakdown of session statuses (approved/rejected/pending/under review) with percentage labels.

Daily Approval Rate (Line Chart): Percentage of sessions approved each day, scaled 0-100%.

Pipeline Stage Latency (Horizontal Bar Chart): Average and P95 response times in milliseconds for each pipeline stage (document_ocr, face_match, deepfake, liveness, final_report).

11.8 Admin Review Queue

The admin review queue (accessible via the admin portal) displays:

- List of all sessions with status `manual_review` or `flagged`.
- Per-session score breakdown.
- Three action buttons: **Approve**, **Reject** (requires reason), **Request Re-Upload**.
- On approval, the session status and user KYC status are updated to `approved`.
- On rejection, the admin must provide a rejection reason, which is stored in the `admin_reviews` table.

Chapter 12

Conclusion and Future Scope

12.1 Achievements

This project successfully designed and implemented a comprehensive AI-based real-time eKYC system with the following key achievements:

1. **Complete Five-Stage Pipeline:** A production-ready verification pipeline covering document OCR, biometric face matching, deepfake detection, liveness verification, and automated decision-making was implemented and demonstrated end-to-end.
2. **State-of-the-Art AI Integration:** The system integrates Google Gemini 2.5 Flash Lite for document understanding, dlib for face extraction, a HuggingFace deepfake detector, and Gradio-hosted face-similarity and liveness models into a unified REST API.
3. **Production-Grade Architecture:** The backend follows clean layered architecture (FastAPI + SQLAlchemy async + Alembic), is containerised with Docker Compose, and exposes a fully documented OpenAPI schema.
4. **Secure Authentication and RBAC:** JWT-based authentication, bcrypt password hashing, and three-tier role-based access control are fully implemented with proper HTTP status codes and dependency injection.
5. **Rich Administrative Tooling:** An admin review queue with manual approve/reject/re-upload workflow, and an analytics dashboard with six KPI cards and four Chart.js visualisations, provide excellent operational visibility.

6. **Webcam Integration:** Real-time selfie capture via the WebRTC MediaDevices API eliminates the need for separate camera apps, providing a seamless browser-based onboarding experience.
7. **Automated Test Suite:** 11 backend (pytest) and 10 frontend (Vitest) tests cover core authentication, security, decision engine, and UI components, all passing.
8. **Compliance-Ready Design:** Audit logging, role-based data access, cascade-delete data management, and configurable data retention align with GDPR and RBI KYC guidelines.

12.2 Limitations

- **External service dependency:** The system's AI capabilities (face matching, deepfake, liveness) depend on third-party hosted services. If these services are unavailable or throttled, the pipeline fails. A self-hosted model deployment would improve resilience.
- **OCR accuracy variability:** Gemini OCR performs well on high-quality scans but struggles with low-contrast or damaged documents. More robust pre-processing and model fallback would improve reliability.
- **No real-time video liveness:** The current liveness check operates on a single static image. True liveness (blink detection, head movement) requires video frame analysis.
- **In-memory pipeline state:** Pipeline stage completion tracking relies on the database session state rather than a distributed cache. This limits horizontal scaling of the backend.
- **Email/SMS notifications:** The email service is a console-logging mock; no actual email or SMS is sent to users upon KYC completion.
- **Document forgery scoring:** The current forgery score is a heuristic (coverage ratio analysis), not a learned model. It has low sensitivity to sophisticated forgery techniques.

12.3 Future Scope

12.3.1 Short-Term Enhancements (0-6 months)

- **Real email/SMS notifications:** Integrate SendGrid or AWS SES for actual email delivery and Twilio for SMS on KYC decision.
- **Self-hosted ML models:** Replace Gradio external calls with locally hosted InsightFace buffalo_s model to eliminate third-party dependency and improve latency.
- **Video-based liveness:** Process uploaded short video clips to perform blink detection and head-pose-based liveness verification.
- **Enhanced forgery detection:** Train a binary classification model on authentic vs. tampered document images to replace the heuristic forgery score.
- **Expanded test coverage:** Add end-to-end API tests for all pipeline stages and security boundary tests.

12.3.2 Medium-Term Enhancements (6-18 months)

- **Mobile application:** Build native iOS and Android apps with camera SDK integration for a more natural onboarding experience.
- **Multi-language OCR:** Extend Gemini prompting to support regional language documents (Hindi, Tamil, Telugu, Bengali) for broader Indian market coverage.
- **Blockchain audit trail:** Store verification session hashes on a blockchain (e.g., Ethereum, Polygon) to create tamper-proof, independently verifiable audit records.
- **Government API integration:** Integrate with UIDAI's Aadhaar OTP verification API and DigiLocker for document provenance verification.
- **Distributed pipeline state:** Use Redis to persist pipeline session state, enabling horizontal scaling of the FastAPI backend behind a load balancer.

12.3.3 Long-Term Enhancements (18+ months)

- **Multimodal biometrics:** Add fingerprint, iris, and voice biometric options for higher-assurance verification, particularly for high-value financial transactions.
- **Explainable AI:** Integrate Grad-CAM heatmaps for deepfake detection (highlight-

ing which image regions triggered the detection) and SHAP values for the decision engine (showing the contribution of each score to the final decision).

- **Federated learning:** Enable privacy-preserving model improvement by training the deepfake and liveness models on encrypted data across participating institutions without centralising sensitive data.
- **Zero-knowledge proofs:** Implement ZKP-based identity verification where the user proves possession of valid credentials without revealing the credentials themselves.
- **Real-time WebRTC streaming:** Replace the asynchronous image upload model with a real-time video stream for fully interactive liveness verification.

12.4 Concluding Remarks

This project demonstrates that a sophisticated, multi-stage AI-powered eKYC system can be designed, implemented, and demonstrated by a single developer using modern open-source frameworks and accessible AI APIs within an academic timeframe. The architecture is production-ready in structure, the security controls are appropriate for a web-facing authentication system, and the user interface provides a professional, seamless onboarding experience.

The system represents a meaningful contribution to the understanding of how AI services can be composed into real-world identity verification pipelines, and provides a solid foundation for any of the future enhancements described above. The combination of FastAPI's async performance, React's component model, Chart.js's visualisation capabilities, and the diverse AI service ecosystem demonstrates the power of API-driven AI integration for building complex, commercially relevant applications.

Bibliography

1. Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). ArcFace: Additive Angular Margin Loss for Deep Face Recognition. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 4690–4699.
2. Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. *Proceedings of the IEEE CVPR*, 815–823.
3. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 5998–6008.
4. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.
5. Chollet, F. (2017). Xception: Deep Learning with Depthwise Separable Convolutions. *Proceedings of the IEEE CVPR*, 1251–1258.
6. Durall, R., Keuper, M., & Keuper, J. (2020). Watch Your Up-Convolution: CNN Based Generative Deep Neural Networks are Failing to Reproduce Spectral Distributions. *Proceedings of the IEEE/CVF CVPR*, 9273–9282.
7. Boulkenafet, Z., Komulainen, J., & Hadid, A. (2017). OULU-NPU: A Mobile Face Presentation Attack Database with Real-World Variations. *IEEE International Conference on Automatic Face and Gesture Recognition (FG)*, 612–618.
8. Liu, Y., Jourabloo, A., & Liu, X. (2018). Learning Deep Models for Face Anti-Spoofing: Binary or Auxiliary Supervision. *Proceedings of the IEEE CVPR*, 389–398.
9. Oquab, M., Darcet, T., Moutakanni, T., Vo, H. V., Szafraniec, M., Khalidov, V., et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. *arXiv preprint arXiv:2304.07193*.

10. Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. *Proceedings of the IEEE ICCV*, 1–11.
11. Ramé, A., Sun, G., & Cord, M. (2022). DICE: Leveraging Sparsification for Out-of-Distribution Detection. *Proceedings of the European Conference on Computer Vision (ECCV)*.
12. Team, FastAPI. (2024). *FastAPI Documentation*. Sebastián Ramírez. Available at: <https://fastapi.tiangolo.com/>
13. Meta AI Research. (2023). *React Documentation*. Available at: <https://react.dev/>
14. SQLAlchemy Authors. (2024). *SQLAlchemy 2.0 Documentation*. Available at: <https://docs.sqlalchemy.org/>
15. King, D. E. (2009). Dlib-ml: A Machine Learning Toolkit. *Journal of Machine Learning Research*, 10, 1755–1758.
16. Reserve Bank of India. (2023). *Master Direction – Know Your Customer (KYC) Direction, 2016 (Updated 2023)*. RBI/DBR/2015-16/18.
17. Google. (2024). *Gemini API Documentation – Vision Language Models*. Available at: <https://ai.google.dev/docs>
18. Hugging Face. (2024). *Inference API Documentation*. Available at: <https://huggingface.co/docs/inference>
19. Abadi, M., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 265–283.
20. OWASP Foundation. (2023). *OWASP Top Ten Web Application Security Risks*. Available at: <https://owasp.org/www-project-top-ten/>

Glossary

API Application Programming Interface – a set of rules and protocols for building and interacting with software applications.

ArcFace A face recognition loss function (Additive Angular Margin Loss) that improves face embedding discriminability.

bcrypt A password-hashing function designed to be computationally expensive and resistant to brute-force attacks.

CORS Cross-Origin Resource Sharing – a browser security feature controlling which domains can access a web API.

deepfake Media (image, video, audio) that has been digitally manipulated using AI to replace a person’s likeness with another.

dlib An open-source machine learning C++ library providing tools for face detection and facial landmark estimation.

DFD Data Flow Diagram – a graphical representation of how data flows through a system.

Docker A platform for developing, shipping, and running applications in lightweight containers.

eKYC Electronic Know Your Customer – a digital process for verifying a customer’s identity online.

embedding A fixed-length numerical vector representing an entity (such as a face) in a high-dimensional space.

FastAPI A modern, fast Python web framework for building APIs with automatic OpenAPI documentation.

GAN Generative Adversarial Network – a class of AI models capable of generating

realistic synthetic images.

Gradio An open-source library for building and sharing ML model demos with web interfaces and APIs.

HuggingFace A platform hosting thousands of pre-trained AI models accessible via a REST Inference API.

JWT JSON Web Token – a compact, URL-safe means of representing claims (such as identity and role) between two parties.

KYC Know Your Customer – a regulatory requirement for financial institutions to verify customer identities.

liveness detection A biometric technique to verify that a presented face belongs to a live person, preventing spoofing with static images.

MongoDB A NoSQL document database that stores data as JSON-like documents.

MVC Model-View-Controller – a software design pattern separating data, presentation, and control logic.

NFC Near Field Communication – a short-range wireless protocol (used in some advanced KYC setups to read electronic document chips).

OWASP Open Web Application Security Project – a nonprofit organisation focused on improving software security.

OCR Optical Character Recognition – technology that converts images of text into machine-readable text.

ORM Object-Relational Mapping – a technique allowing developers to interact with a database using object-oriented code.

RBAC Role-Based Access Control – access restrictions based on a user's assigned role.

RBI Reserve Bank of India – India's central banking authority, which mandates KYC compliance for financial institutions.

React A JavaScript library for building user interfaces using reusable components.

Redis An in-memory data structure store used as a database, cache, and message broker.

REST Representational State Transfer – an architectural style for distributed hypermedia systems.

SQLAlchemy A Python SQL toolkit and ORM providing full SQL expression and database access capabilities.

SPA Single-Page Application – a web application that loads a single HTML page and dynamically updates content.

SRS Software Requirements Specification – a document describing the functional and non-functional requirements of a system.

TLS Transport Layer Security – a cryptographic protocol for securing communications over a network.

TypeScript A strongly typed superset of JavaScript that compiles to plain JavaScript.

UUID Universally Unique Identifier – a 128-bit identifier standard used as database primary keys.

V-CIP Video Customer Identification Process – RBI-mandated video-based KYC verification.

Vite A build tool for modern web projects providing fast development server and production bundling.

VLM Vision-Language Model – an AI model that processes both image and text inputs.

WebRTC Web Real-Time Communication – a browser API enabling real-time audio, video, and data sharing.

Appendix A

API Endpoint Catalog

Table A.1: Complete REST API Endpoint Reference

Method	Path	Auth	Role	Description
POST	/api/v1/auth/register	No	Any	Register a new user
POST	/api/v1/auth/login	No	Any	Login and get JWT
GET	/api/v1/auth/me	Yes	Any	Get current user profile
POST	/api/v1/features/document-ocr	Yes	Any	Upload document + run OCR
POST	/api/v1/features/face-match	Yes	Any	Submit selfie + run face match
POST	/api/v1/features/deepfake	Yes	Any	Run deepfake detection
POST	/api/v1/features/liveness	Yes	Any	Run liveness detection
POST	/api/v1/features/final-report	Yes	Any	Generate final decision + PDF
GET	/api/v1/features/report/{id}	Yes	Any	Download PDF report
GET	/api/v1/features/images/{key}	Yes	Any	Retrieve protected image
GET	/api/v1/analytics/overview	Yes	Admin/Rev	KPI overview counts
GET	/api/v1/analytics/trends	Yes	Admin/Rev	30-day daily trend data

Table A.1 – continued

Method	Path	Auth	Role	Description
GET	/api/v1/analytics/latency	Yes	Admin/Rev	Stage latency metrics
GET	/api/v1/admin/reviews	Yes	Admin/Rev	List flagged sessions
POST	/api/v1/admin/reviews/{id}	Yes	Admin/Rev	Submit review decision
GET	/api/v1/admin/audit-logs	Yes	Admin	Query audit log entries
GET	/api/v1/system/health	No	Any	Service health check

Appendix B

Database Schema Summary

Table B.1: users Table Schema

Column	Type	Constraints	Description
id	UUID	PK, default gen	Unique user identifier
full_name	VARCHAR(255)	NOT NULL	Full legal name
email	VARCHAR(255)	UNIQUE	Email address
phone	VARCHAR(20)	UNIQUE	Phone number
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
role	VARCHAR(20)	CHECK	user/admin/reviewer
kyc_status	VARCHAR(30)	CHECK	pending/approved/rejected/under_review
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Last update timestamp

Table B.2: verification_sessions Table Schema

Column	Type	Constraints	Description
id	UUID	PK	Session identifier
user_id	UUID	FK users	Owner user
document_id	UUID	FK documents	Reference document
live_image_path	VARCHAR		Selfie file path
live_face_crop_path	VARCHAR		Extracted live face path
face_match_score	FLOAT	CHECK 0–100	Face similarity score
deepfake_score	FLOAT	CHECK 0–100	Deepfake probability
liveness_score	FLOAT	CHECK 0–100	Liveness confidence
final_score	FLOAT	CHECK 0–100	Composite decision score
status	VARCHAR(20)	CHECK	pending/approved/manual_review/reject
report_path	VARCHAR		Generated PDF path
created_at	TIMESTAMPTZ	DEFAULT	Creation timestamp

Appendix C

Technologies and Library Versions

Table C.1: Backend Technology Stack

Component	Library/Tool	Version
Language	Python	3.12
Web Framework	FastAPI	0.115+
ASGI Server	Uvicorn	0.30+
ORM	SQLAlchemy	2.0 (async)
Database Migration	Alembic	1.13+
Auth	python-jose	3.3+
Password Hashing	passlib[bcrypt]	1.7+
Validation	Pydantic	2.x
Rate Limiting	slowapi	0.1+
OCR	google-genai	1.x
Face Detection	dlib	19.24
Image Processing	OpenCV (cv2)	4.x
Deepfake Detection	htpx (HF API)	0.27+
Face Services	gradio-client	1.x
PDF Generation	WeasyPrint	62+
Audit Logging	motor (MongoDB async)	3.x
Caching	redis (async)	5.x
Testing	pytest, pytest-asyncio	8.x
Local Database	SQLite	(built-in)
Production Database	PostgreSQL	15

Table C.2: Frontend Technology Stack

Component	Library/Tool	Version
Language	TypeScript	5.6
UI Framework	React	18
Build Tool	Vite	5
Charts	Chart.js via react-chartjs-2	4.x
HTTP Client	Fetch API	(built-in)
State Management	React Hooks (useState, useEffect)	(built-in)
Webcam	MediaDevices API (getUserMedia)	(browser)
Testing	Vitest + @testing-library/react	2.x
Styling	Custom CSS (no framework)	–

Table C.3: Infrastructure and Deployment Stack

Component	Tool	Version
Containerisation	Docker	26+
Orchestration	Docker Compose	2.x
Reverse Proxy	Nginx	1.25+
Structured DB	PostgreSQL	15
Audit DB	MongoDB	6
Cache / Rate Limit	Redis	7
Version Control	Git	2.x
Repository	GitHub	–